



AWS Academy Cloud Developing
Module 07 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

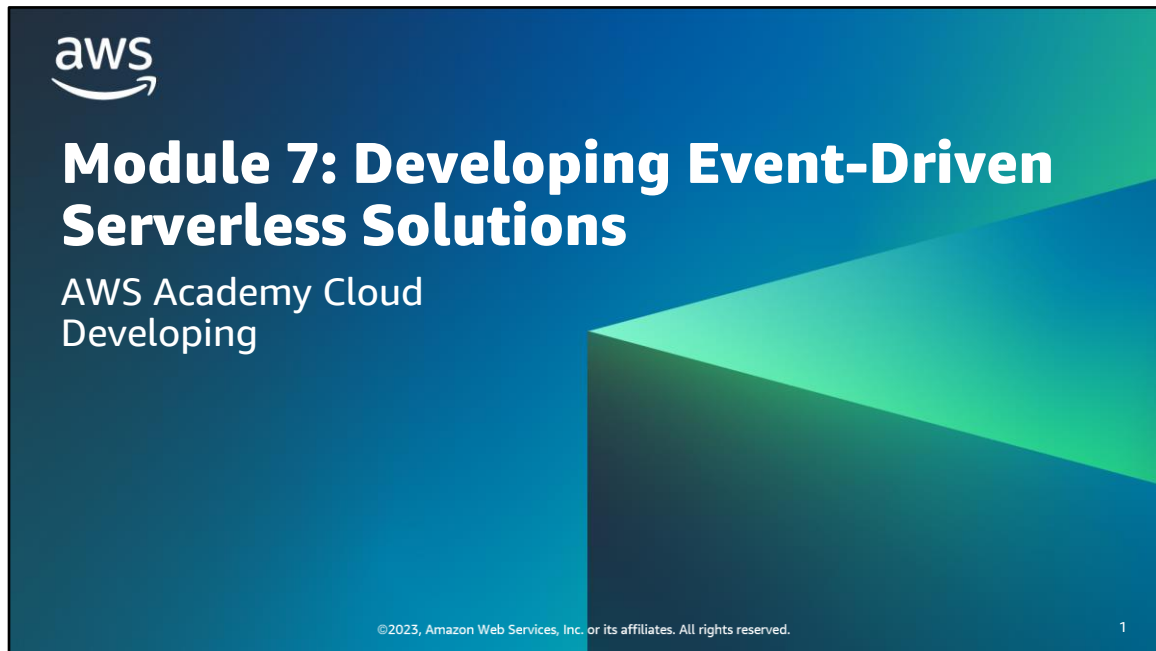
© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

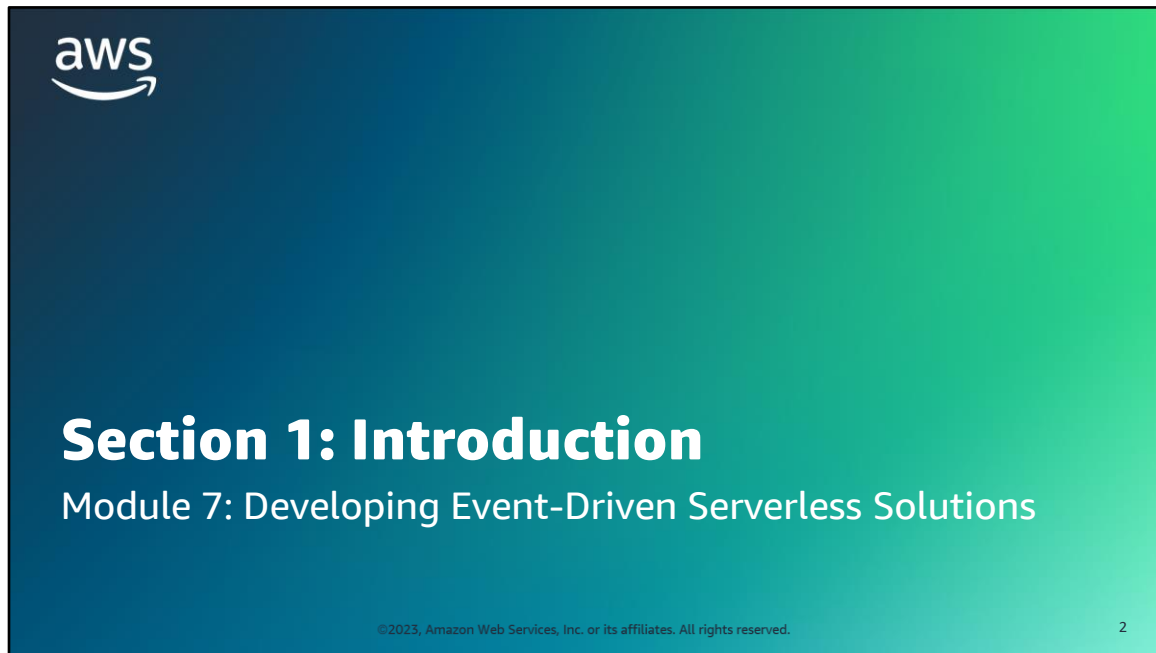
All trademarks are the property of their owners.

Contents

Module 7: Developing Event-Driven Serverless Solutions	4
--	---



Welcome to Module 7: Developing Event-Driven Serverless Solutions.



Section 1: Introduction.

Module objectives



At the end of this module, you should be able to:

- Explain serverless computing
- Describe how AWS Lambda works
- Recognize Lambda invocation models
- Identify how to use AWS Identity and Access Management (IAM) to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why AWS X-Ray is a critical developer tool
- Create Lambda functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to:

- Explain serverless computing
- Describe how Lambda works
- Recognize Lambda invocation models
- Identify how to use AWS Identity and Access Management (IAM) to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why AWS X-Ray is a critical developer tool
- Create Lambda functions

Module overview

Sections

1. Introduction
2. Introducing serverless computing
3. Introducing Lambda
4. Invoking Lambda functions
5. Setting permissions for Lambda
6. Authoring and configuring Lambda functions
7. Deploying Lambda functions
8. Monitoring and debugging tools for application developers

Demonstration

Using X-Ray with Lambda

Lab

Creating Lambda Functions Using the AWS SDK for Python



Knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

1. Introduction
2. Introducing serverless computing
3. Introducing AWS Lambda
4. Invoking Lambda functions
5. Setting permissions for Lambda
6. Authoring and configuring Lambda functions
7. Deploying Lambda functions
8. Monitoring and debugging tools for application developers

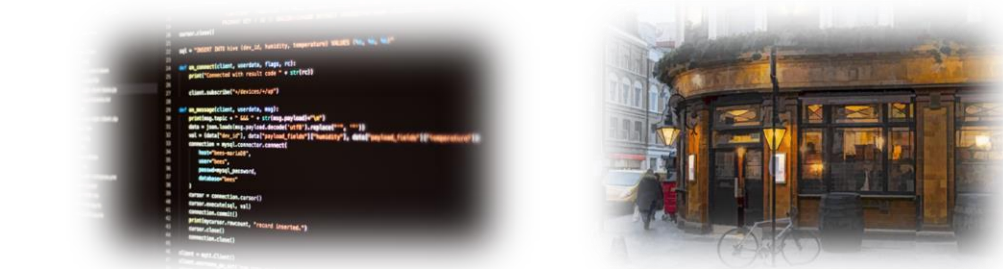
This module also includes:

- A demonstration of using AWS X-Ray with a Lambda function.
- A lab where you create a Lambda function to query a database, call a deployment package, and configure a Lambda function. You then replace a mock endpoint with your Lambda function.

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Sofía finished using the mock endpoints to test the API, and now she wants to launch the dynamic version of the café website. She needs to update the API integration to connect to the backend database.

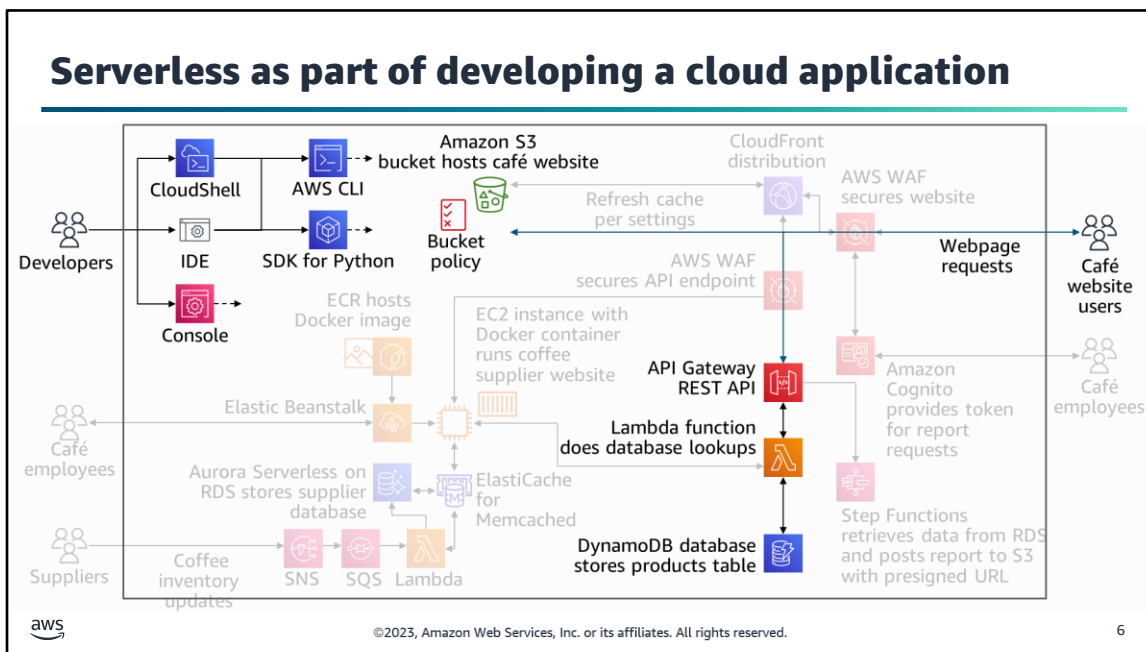


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

5

Sofía finished using the mock endpoints to test the API, and now she wants to launch the dynamic version of the café website. She needs to update the API integration to connect to the backend database.

Sofía will replace the mock endpoints with functional endpoints, and use a Lambda function to bridge the connection between the APIs and the data stored in DynamoDB.

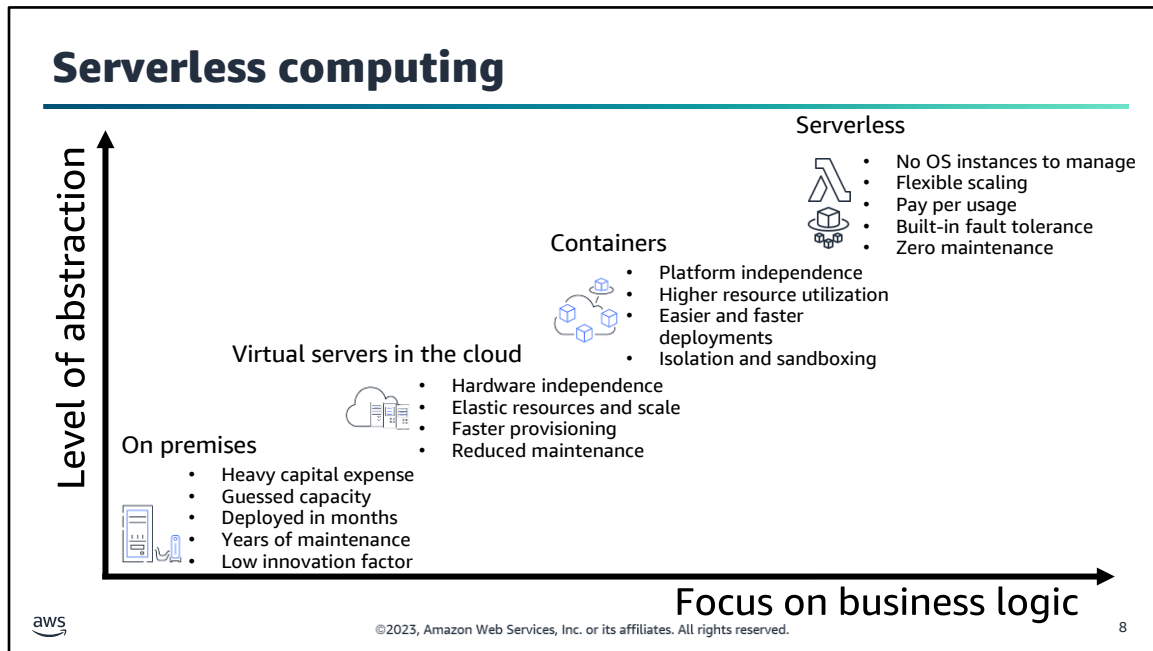


The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will configure calls to the API Gateway products endpoints to invoke a Lambda function that queries and returns data from DynamoDB.



Section 2: Introducing serverless computing.



One benefit of cloud computing is that you can abstract the infrastructure layer to alleviate many operational tasks. Thus, you can focus more on the business logic for your applications. You do not need to manage instances, operating systems, or servers. Everything that is required to run and scale your application with high availability is handled for you. Serverless computing eliminates infrastructure management tasks, such as server or cluster provisioning, patching, operating system maintenance, and capacity provisioning. The reduced overhead also means you can experiment and innovate faster.

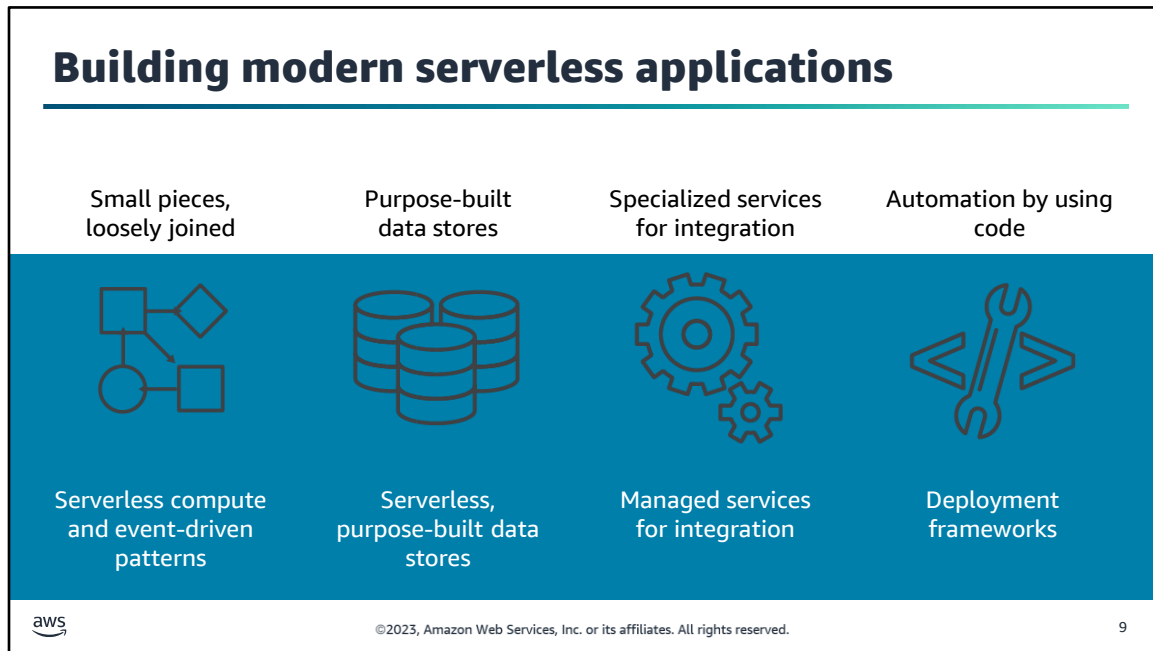
You don't pay for any infrastructure when the code isn't running. Additionally, with serverless computing, your code runs only when it's needed.

In a practical sense, with servers you must think about the following:

- Configuring instances
- Updating your OS
- Installing the application platform
- Configuring automatic scaling and load balancing
- Building and deploying applications
- Continuously patching, securing, and monitoring servers
- Monitoring and maintaining applications

With serverless, you can focus on the following:

- Building and deploying applications
- Monitoring and maintaining applications



From a builder's perspective, the abstraction of infrastructure into the cloud from virtual servers to containers and now to serverless has changed application development. It coincides with a move away from monolithic applications and a move toward service-oriented architectures and microservices.

The ideas of distributed applications and microservices aren't new, but the evolution of cloud services makes it easier to build applications that follow these principles:

- Use small, loosely joined pieces and purpose-built data stores. This strategy provides the flexibility to make changes without affecting other parts of the system, scale and deploy pieces independently, and incorporate reversible decisions.
- Use specialized integration services to increase the speed of delivery and decrease the effort of integrations between services.
- Write infrastructure as code to automate as much of the deployment process as you can. This approach makes it simpler to replicate environments, and deploy more frequently with minimal overhead and greater confidence.

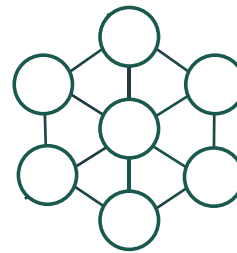
Serverless ("Serverless on AWS" at <https://aws.amazon.com/serverless/>) is an efficient approach to modern applications:

- With serverless compute services, you can focus on writing business logic while someone else handles scaling and high availability.
- With purpose-built serverless databases, you can choose the features and capacity that suit the type and volume of data that each service needs.
- Managed services provide API proxy, messaging, and orchestration and have prebuilt integrations with Lambda and other AWS services. With this process you can connect services by using asynchronous event-driven patterns to achieve decoupling and flexibility. At the same time, it reduces the undifferentiated lifting of building the integrations to connect your components.
- Serverless deployment frameworks make it easier to use infrastructure as code for deployments and automate the deployment pipeline.

Moving away from a monolithic application



Monolith
Does everything



Microservice
Does one thing



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

10

With a monolithic application, things start out easier. It's simpler to build and test, and you have only one artifact to deploy. For some less complex applications, a monolithic approach might be the best option for a long time.

For complex applications that grow over time, the interdependencies among components and the necessary coordination among development teams for testing and deploying become more difficult. These concerns can decrease developer productivity.

- You have one technology stack in which you must develop. You must hire and train developers in that technology and can't take advantage of skills in other languages. Any change to the technology stack becomes a major project.
- You have a single database. The most demanding use case drives your cost and capacity. Operations that aren't suited to the database or data structures will continually be tuned to strive for better performance. However, they will reach a limit that is impossible to improve on without a redesign.

From an operational standpoint, many developers are pushing changes through a shared release pipeline, which causes friction at many points of the lifecycle.

- If you want to upgrade a shared library to use a new feature, you must convince everyone else to upgrade at the same time.
- If you want to quickly push an important fix for your feature, you still must merge it with everyone else's in-process changes. This approach leads to merge Fridays or, worse yet, merge weeks when all developers compile their changes and resolve any conflicts for the next release.
- After development, you also face overhead when you're pushing the changes through the delivery pipeline. You must rebuild the entire application, run all of the test suites to ensure that you have no regression issues, and redeploy the entire application.

Every new feature adds to the complexity of the coordination, deployment, and ongoing operations effort. Technical debt stacks up, and it becomes nearly impossible to untangle code.

As an example, before migrating to a microservices approach, Amazon had a central team whose sole job was to deploy its monolithic application into production.

With a modern serverless application, the effort to get started is higher. Moving to microservice-based development requires a significant surge in effort as everyone learns and make mistakes. Often, everyone is new to the approaches that are required. Moving from experiments and utilities into full-scale production applications requires a surge of effort that includes both technology and organizational changes. The energy that it takes to get over that initial hurdle can hold organizations back without some initiating event or tipping point.

However, after the initial learning curve, the agility that this approach provides starts to accelerate the speed with which you can release new features. It also reduces the effort that it takes to keep them running.

Microservices provide these benefits:

- Scale individual services independently.
- Choose the best technology stack and data store for each service.
- Deploy changes independently as often as desired.
- Focus on a specific service and innovate within that service.



Section 3: Introducing Lambda.

AWS Lambda



AWS Lambda

Lambda is a serverless compute service that you can use to run code without provisioning or managing servers.

- Invokes your code in response to events
- Scales automatically
- Provides built-in code monitoring and logging with Amazon CloudWatch



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

12

AWS Lambda is the compute service for serverless.

Lambda has the following advantages:

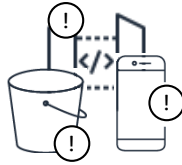
- Lets you run code without provisioning or managing servers
- Invokes your code in response to events that you configure
- Scales automatically based on demand
- Incorporates built-in monitoring and logging with Amazon CloudWatch

For more information about Lambda, see “AWS Lambda” at <https://aws.amazon.com/lambda/>.

How Lambda works



Upload your code to Lambda or write code in the Lambda editor



Set up your code to run when events occur in other AWS services, at HTTP endpoints, or as part of in-app activity



Lambda runs your code only when it is activated by an event and uses only the compute resources that are needed



Pay only for the compute time that you use



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

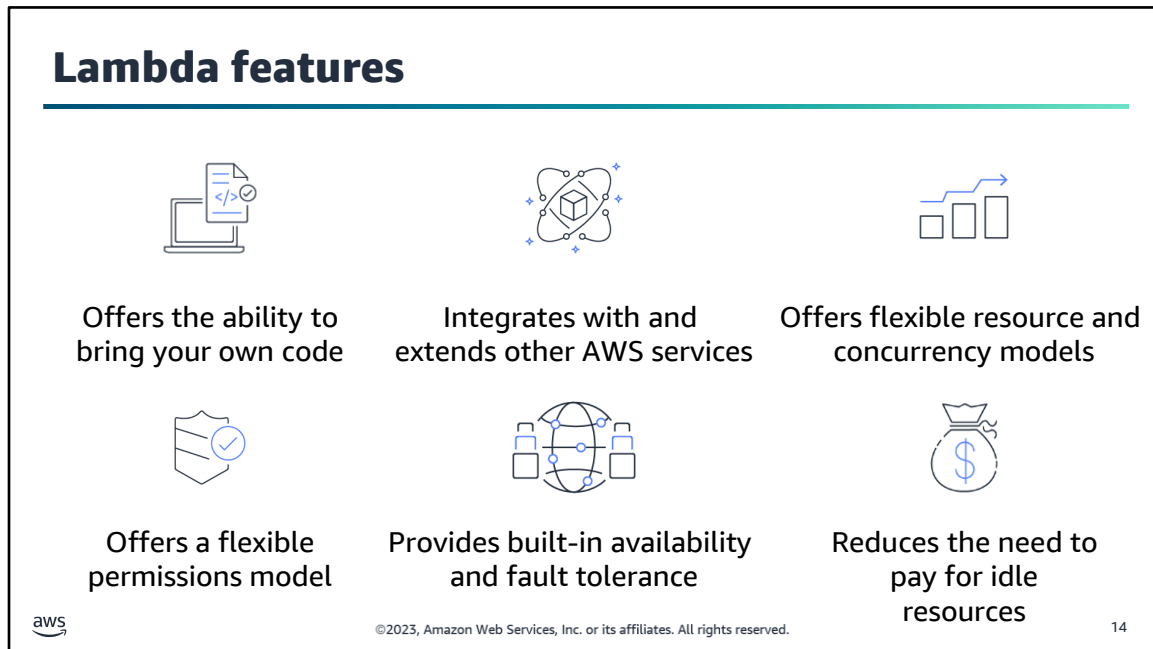
13

Functions and event sources are the core components of Functions and event sources are the core components of Lambda. An *event source* is the entity that publishes events to Lambda. A Lambda *function* is the custom code that you provide, which processes the events. Lambda runs your function on your behalf.

To use Lambda, start by uploading existing code or writing code in the Lambda editor.

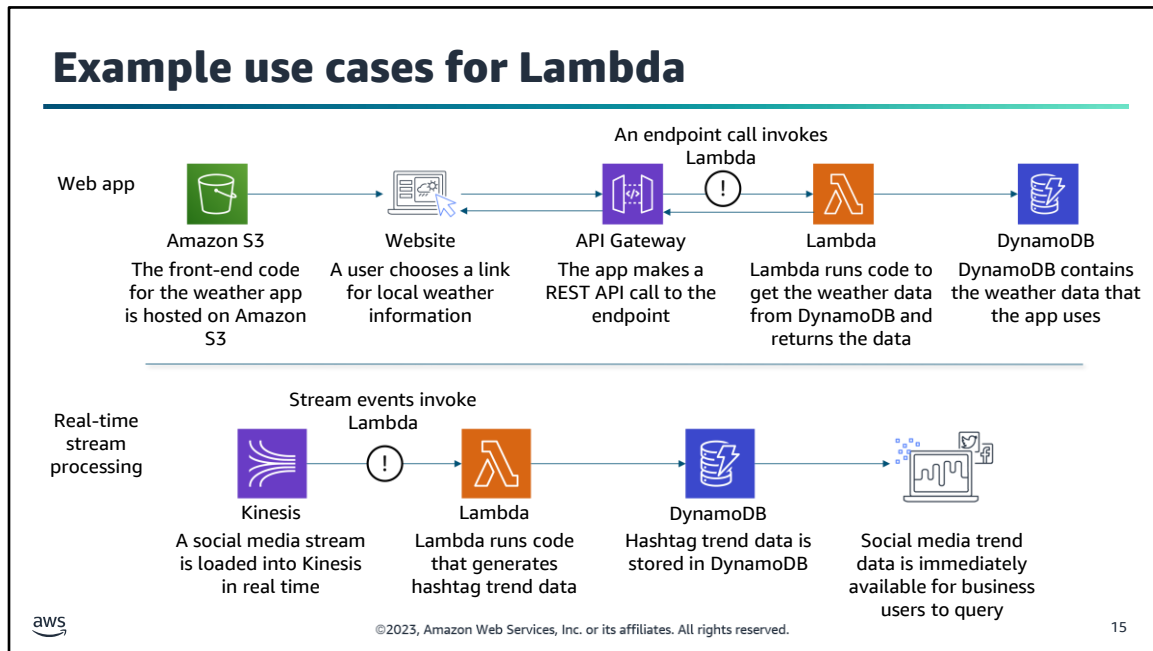
Then, set up your functions to run when events occur in other AWS service. Functions can also respond to incoming HTTP requests, consume events from a queue or stream, or run on a schedule.

Lambda runs your code only when activated and uses only the necessary compute resources. You pay only for the compute time that you use.



AWS Lambda provides the following features:

- Offers the ability to bring your own code – The code that you write for Lambda isn't written in a new language that you have to learn. Development in Lambda is not tightly coupled to AWS, so you can easily port code in and out of AWS. With Lambda, you bring your code (written in your language of choice—such as Node.js, Java, Python, C#, Go, or Ruby), custom runtimes, and libraries.
- Integrates with and extends other AWS services – From within your Lambda function, you can do anything that traditional applications can do. Examples include calling an AWS SDK or invoking a third-party API.
- Offers flexible resource and concurrency models – Lambda scales in response to events. You configure memory settings, and AWS handles the details, such as CPU, network, and I/O throughput.
- Offers a flexible permissions model – Lambda uses AWS Identity and Access Management (IAM) to securely grant access to your resources and give you fine-grained control for invoking your functions.
- Provides built-in availability and fault tolerance – Because Lambda is a fully managed service, high availability and fault tolerance are integrated into the service, without any additional configuration on your part.
- Reduces the need to pay for idle resources – Lambda functions run only when they are invoked, so you don't pay for idle capacity.



You can use Lambda in various ways. Web applications and real-time stream processing are two common use cases. Additional common use cases include the following:

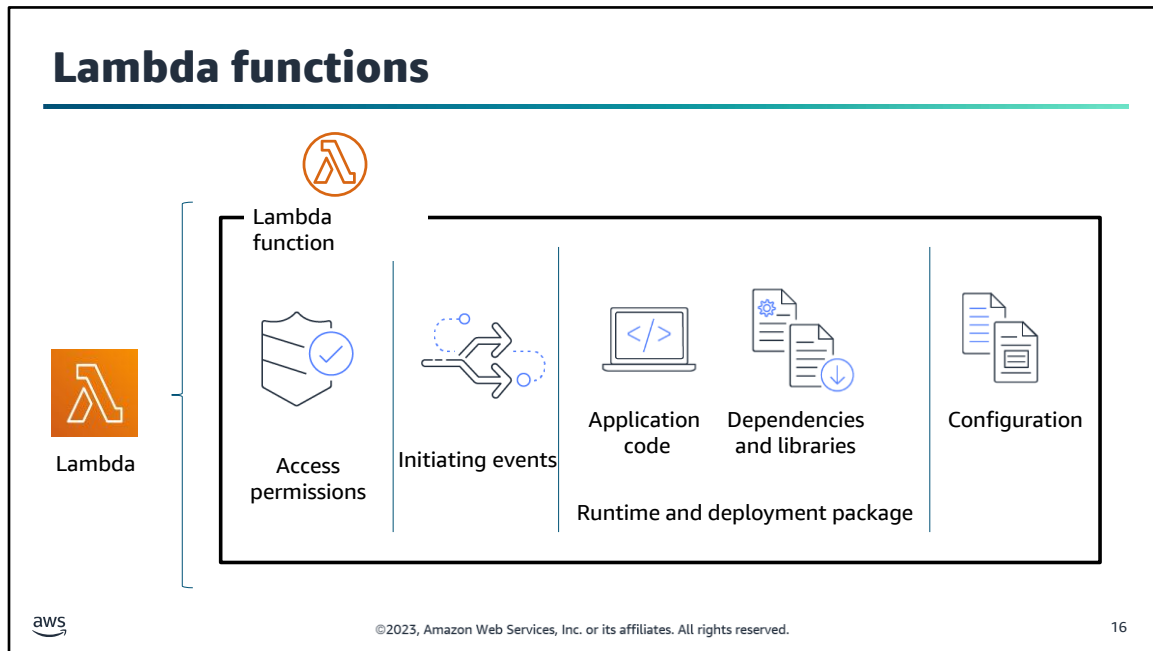
- **Backends** – You can build serverless backends by using Lambda to handle web, mobile, Internet of Things (IoT), and third-party API requests. You can build backends by using Lambda and Amazon API Gateway to authenticate and process API requests. Lambda makes it easy to create rich, personalized application experiences.
- **Data processing** – You can use Lambda to run code in response to triggers, such as changes in data, shifts in system state, or actions by users. AWS services, such as Amazon Simple Storage Service (Amazon S3) and Amazon DynamoDB, can directly invoke Lambda. Other services that can invoke Lambda include Amazon Kinesis, Amazon Simple Notification Service (Amazon SNS), and CloudWatch. AWS Step Functions can also orchestrate Lambda in workflows. You can use Lambda with these services to build various real-time processing systems for serverless data.
- **Chatbots** – Amazon Lex is a service for building conversational interfaces into any application by using voice and text. Amazon Lex uses the same deep learning technologies that power Amazon Alexa, and it integrates with Lambda.
- **Amazon Alexa** – You can use Lambda to build the backend for custom Alexa skills.
- **IT automation** – You can use Lambda to automate repetitive processes by invoking them with events or by running them on a fixed schedule. You can automatically update the firmware of hardware devices, start and stop Amazon Elastic Compute Cloud (Amazon EC2) instances, or schedule security group updates. In addition, you can automate your test and deployment pipeline.

For more case studies, see “AWS Lambda” at <https://aws.amazon.com/lambda/>.

Anything that you want to do with servers you can probably do with Lambda. Use Lambda and serverless to reimagine your approach to new or updated applications.

To get started, follow these suggestions:

- Look for opportunities to introduce serverless through common use cases.
- Choose something authentic, iterate on it, learn from it, and expect to keep working on it in production.
- Expect to change your mind about what you thought was the best solution when you started. Keep learning and updating your approach.



As a developer, you create Lambda functions that the Lambda service manages. When you create a function, you define the permissions for the function and specify which events will invoke the function. You choose a runtime and create a deployment package that includes your application code and any dependencies and libraries necessary to run your code. Finally, you configure parameters such as memory, timeout, and concurrency. When your function is invoked, Lambda provides a runtime environment based on the runtime and configuration options that you selected.

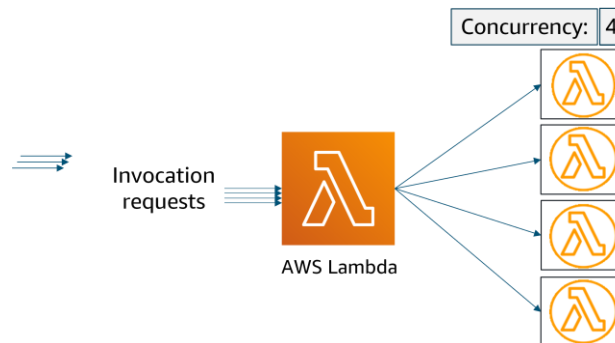
Lambda initializes concurrent environments to maintain the pace of invocation requests up to certain quotas. Each environment is temporary, and the Lambda service controls it, which affects some of the best practices that you should apply to your function code. You will discuss these ideas later in this section.

Lambda supports multiple languages through the use of runtimes, including Node.js, Python, Ruby, Java, Go, and .NET. You can also implement a custom runtime if you want to use another language in Lambda.

For more information about the Lambda runtime environment, see “Lambda Runtimes” at <https://docs.aws.amazon.com/lambda/latest/dg/current-supported-versions.html>.

Lambda functions run in on-demand ephemeral environments

Concurrency: The number of function invocations that are running at one time



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

17

Each invocation of a Lambda function runs in an ephemeral environment—one that is initialized on demand, lasts a brief time, and then is removed. Lambda manages creating and tearing down these environments for your functions. You don't have direct control over these environments.

Concurrency is the number of Lambda function invocations that are running at one time. Lambda provides the environments on demand as it receives requests, which is how Lambda quickly scales horizontally.

Factors that influence concurrency include the following:

- **Reserved concurrency** on a function – This optional value is set per function and reserves a subset of the Regional quota for the function. It also establishes the maximum concurrent instances that are permitted for the function. For more information about managing reserved concurrency, see “Configuring reserved concurrency” at <https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>.
- **Regional quota** – This quota is the total number of invocations that can run concurrently across all Lambda functions within an account by Region. AWS sets this number as a soft quota on the account. For more information about quotas, see “Lambda Quotas” at <https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html>.
- **Burst quota** – Each Region has a burst limit, which prevents concurrency from increasing too quickly if a large spike of requests occurs in a short time period. You cannot modify this limit. For more information about bursts, see “Lambda function scaling” at <https://docs.aws.amazon.com/lambda/latest/dg/invocation-scaling.html>.
- **Request rate and function duration** – This value is the rate at which new invocation requests for the function arrive, in conjunction with how long Lambda takes to run the function.
- **The event source** – Lambda manages concurrency of requests in different ways

based on the type of event that invokes it.

Section 3 key takeaways



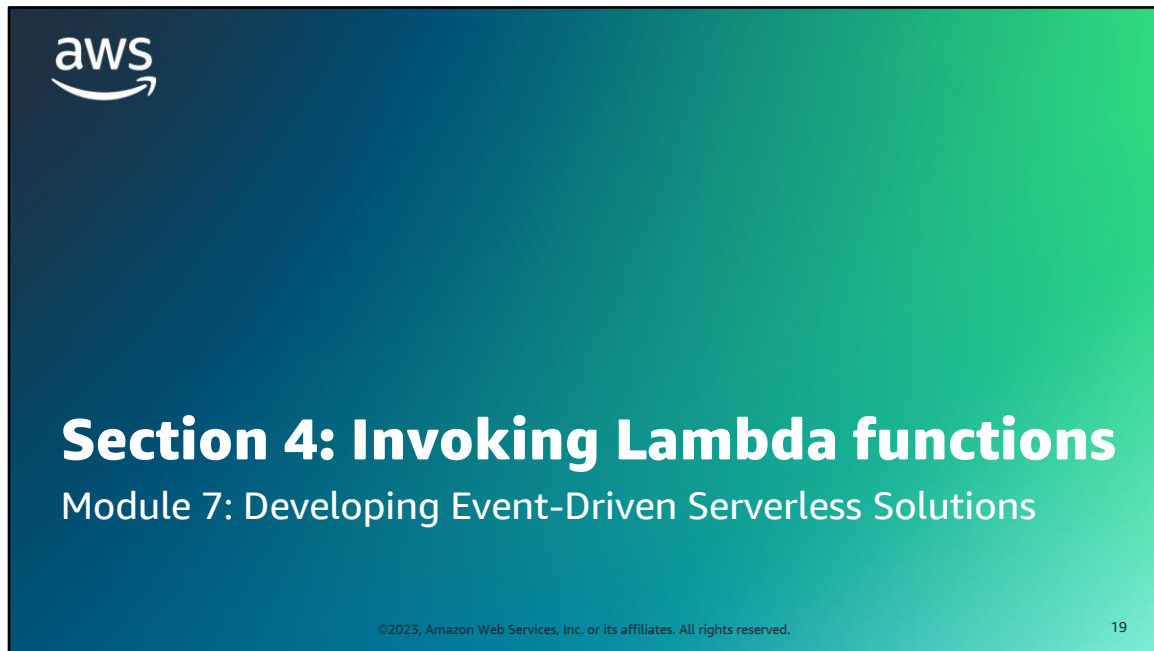
- Lambda is a serverless compute service that integrates with other AWS services.
- You can use Lambda for many use cases including web applications, backends, and data processing.
- Lambda functions run in on-demand temporary environments.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

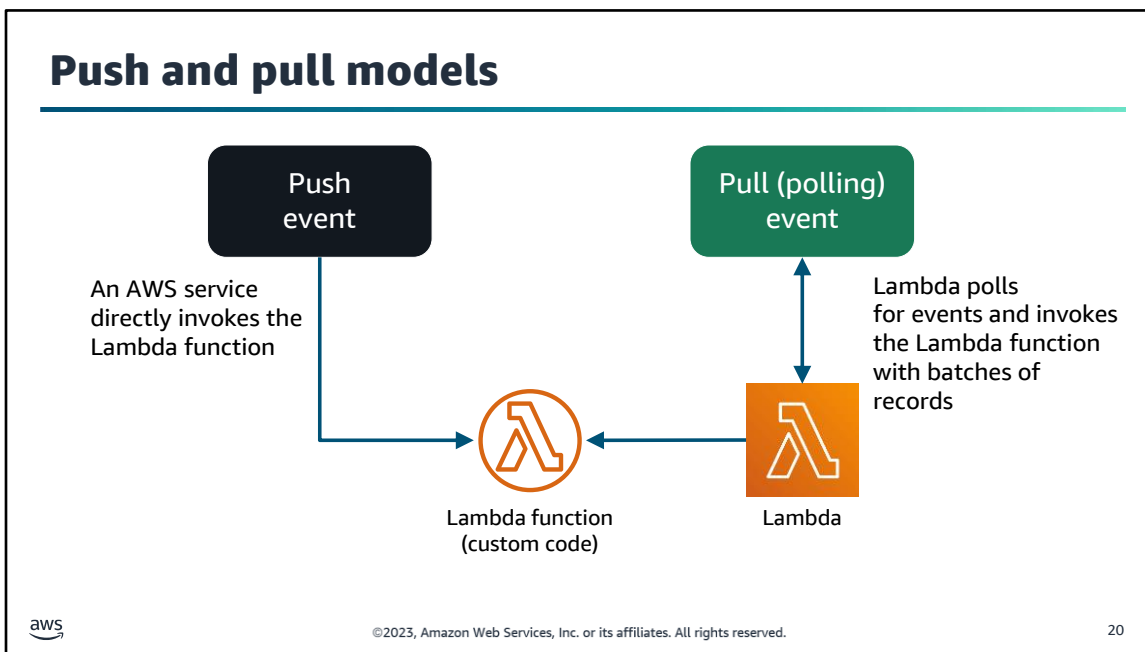
18

Some key takeaways from this section of the module include the following:

- Lambda is a serverless compute service that integrates with other AWS services.
- You can use Lambda for many use cases including web applications, backends, and data processing.
- Lambda functions run in on-demand temporary environments.

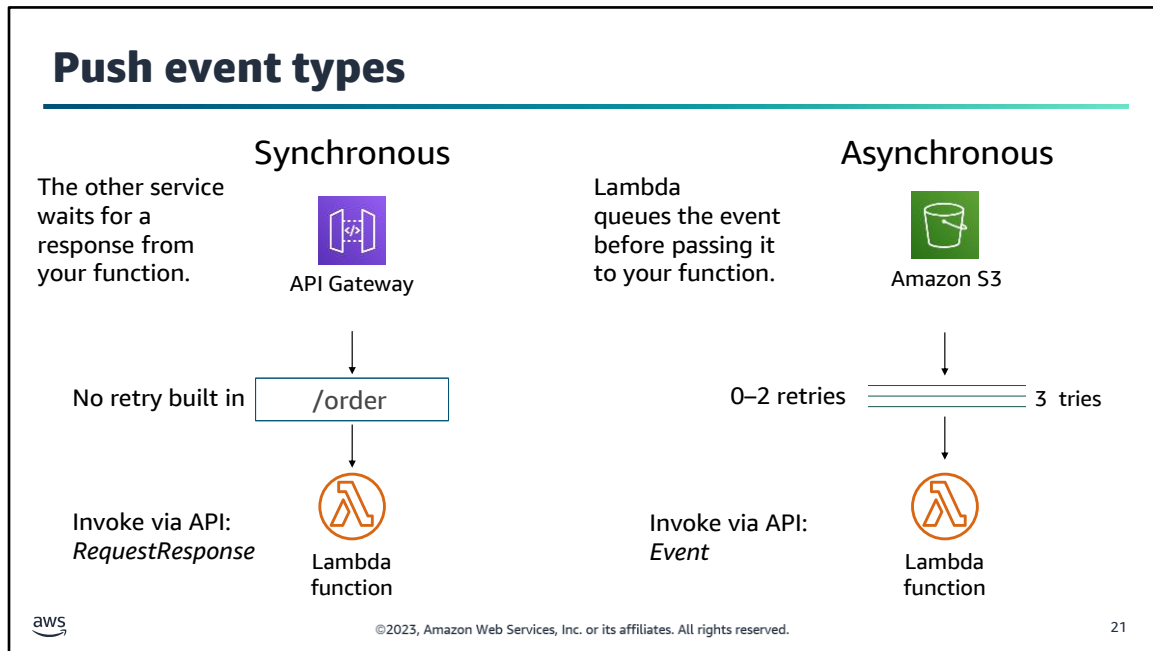


Section 4: Invoking Lambda functions.



Each event source invokes a Lambda function by using either a push or a pull model:

- *Push model* – An event source directly invokes the Lambda function when the event occurs.
- *Pull (or polling) model* – An event source puts information into a stream or queue. Lambda polls the stream or queue and invokes your Lambda function when it detects an event.



Some services invoke your function directly.

For *synchronous* invocation, the other service waits for the response from your function. Consider an example where API Gateway is the event source. In this case, when a client makes a request to your API, that client expects a response immediately. This model has no built-in retry in Lambda. You must manage your retry strategy in your application code.

Examples of synchronous event sources include the following:

- Elastic Load Balancing
- Amazon Cognito
- Amazon Lex
- Amazon Alexa
- Amazon API Gateway
- AWS CloudFormation
- Amazon CloudFront
- Amazon Kinesis Data Firehose

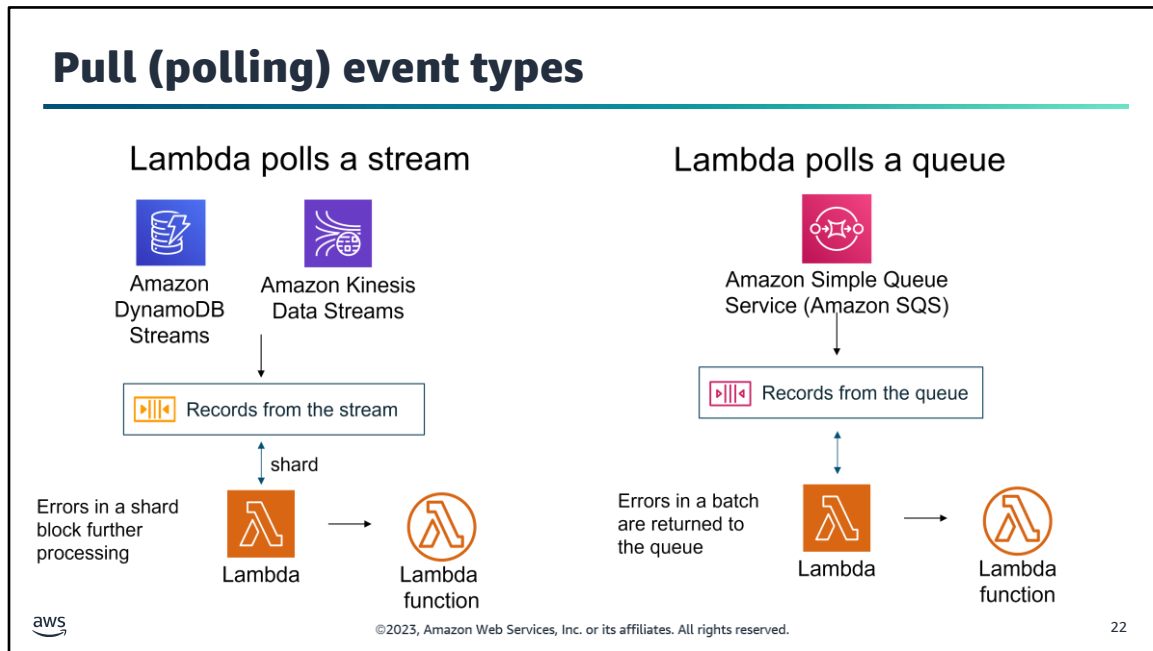
For *asynchronous* invocation, Lambda queues the event before passing it to your function. The other service gets a success response as soon as the event is queued and isn't aware of what happens afterwards. The event sources "sends it then forgets it". If an error occurs, Lambda will automatically retry the invocation twice. It can send failed events to a dead-letter queue that you configure.

Examples of asynchronous event sources include the following:

- Amazon S3
- Amazon SNS
- Amazon Simple Email Service (Amazon SES)
- AWS CloudFormation
- Amazon CloudWatch Logs
- Amazon CloudWatch Events
- AWS Config

In both cases, you invoke your Lambda function by using the *invoke* operation, and you can specify the invocation type as synchronous or asynchronous. When you use an AWS services as a trigger, the invocation type is predetermined for each service. You have no control over the invocation type that these event sources use when they invoke your Lambda function.

For more information how to invoke Lambda functions, see “Invoking Lambda functions” at <https://docs.aws.amazon.com/lambda/latest/dg/invoking-lambda-functions.html>.



In the polling model, Lambda uses event source mappings to get information from a stream or queue. Lambda polls the stream or queue. If it finds records, it delivers the payload and invokes the Lambda function. In this model, the Lambda service itself is extracting data from the stream or queue for processing by the Lambda function.

- Stream-based event sources include Amazon DynamoDB Streams and Amazon Kinesis Data Streams. Stream records are organized into shards. Lambda polls the stream for records and attempts to invoke the function. If a failure occurs, Lambda will not read any new shards until the failed batch of records expires or is processed successfully. You can configure error handling options to modify how Lambda deals with errors.
- Queue-based event sources include Amazon Simple Queue Service (Amazon SQS). Lambda polls the queue for records. If the invocation fails or times out, then the failed message is returned to the queue. Lambda continues to retry the failed message until it's processed successfully or the retry limit or message retention period is reached. Failed messages can be sent to an OnFailure destination or a dead-letter queue.

You can create a mapping between an event source and your Lambda function. Lambda reads items from the event source and invokes the function. For more information about the `CreateEventSourceMapping` operation, see “`CreateEventSourceMapping`” at

https://docs.aws.amazon.com/lambda/latest/dg/API_CreateEventSourceMapping.html.

Polling does not incur any charge. You are charged only if actions result from a poll.

With streams, Lambda retries until it succeeds or the record reaches a 24-hour expiration. Thus, an erroring record can block processing. You can configure error handling options to bypass failing records and send them to an OnFailure destination for offline processing. For more information about how Lambda works with streams, see “Using AWS Lambda with Amazon Kinesis” at <https://docs.aws.amazon.com/lambda/latest/dg/with-kinesis.html>.

With queues, failed records become visible on the queue to be resent to your Lambda function up to a number of retries, which you can configure. You can send records that continue to fail to a dead-letter queue. For more information about how Lambda works with queues, see “Using AWS Lambda with Amazon SQS” at <https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html>.

Invoking a function synchronously

Invoke

```
aws lambda invoke \  
--function-name my-function \  
--payload '{ "name": "Bob" }' \  
response.json
```

Response

```
{"ExecutedVersion": "$LATEST","StatusCode": 200}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

This example shows the command line code to invoke a function synchronously and illustrates the format of the response that Lambda would return.

Notice that with a synchronous event, the acknowledgement is the 200 status.

Invoking a function asynchronously

Invoke

```
aws lambda invoke \  
--function-name my-function \  
--invocation-type Event \  
--payload '{ "name": "Bob" }' \  
response.json
```

Ack

```
{"StatusCode": 202}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

This example shows the command line code to invoke a function asynchronously and illustrates the format of the response that Lambda would return. Notice that with an asynchronous event, the acknowledgement is the 202 status, which indicates that the request was received.

Section 4 key takeaways



- Lambda invokes Lambda functions by using one of these invocation models:
 - Synchronous
 - Asynchronous
 - Polling (event source mapping)
- Lambda has built-in handling that differs by event source type.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

25

Some key takeaways from this section of the module include:

- AWS Lambda uses either a push or pull model for invocation
- Push – Can be synchronous or asynchronous
- Pull – Can be stream-based or non-stream-based



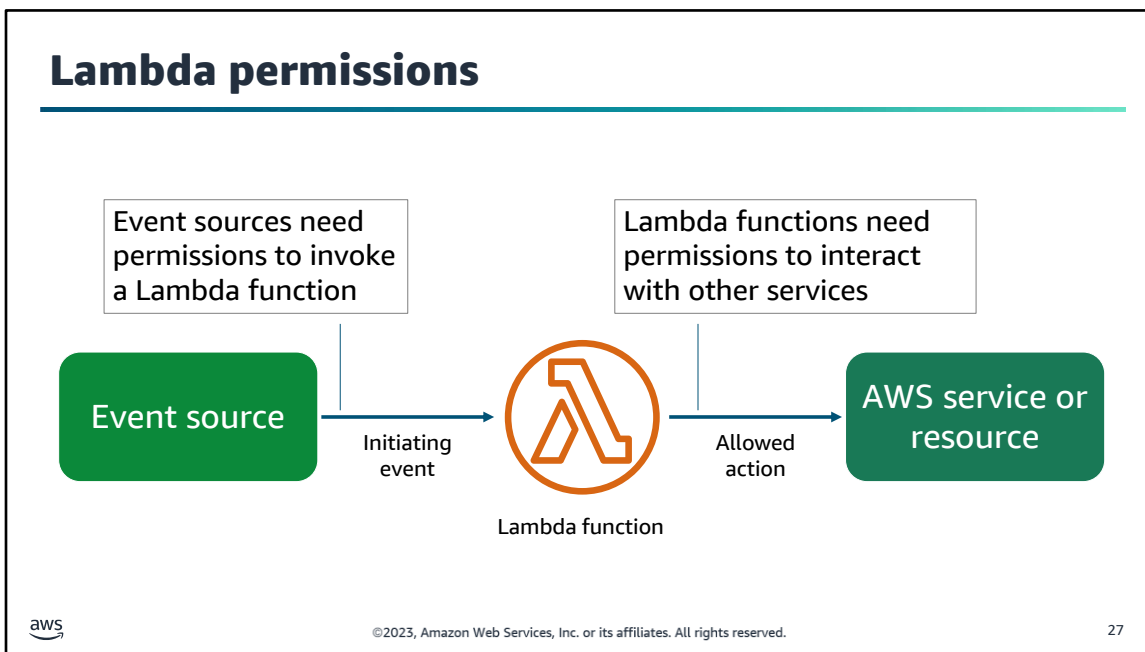
Section 5: Setting permissions for Lambda

Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

26

Section 5: Setting permissions for Lambda.



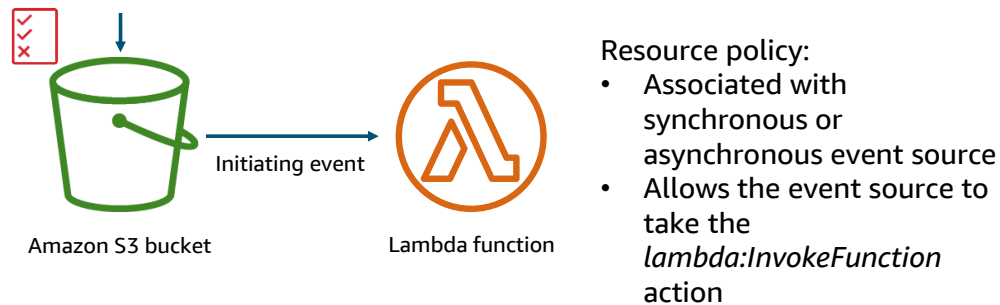
Lambda requires two types of permissions. First, event sources need permissions to invoke a Lambda function. Next, Lambda functions need permissions to interact with other AWS services and resources.

Permissions are handled through IAM.

For more information, see “Lambda resource access permissions” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-permissions.html>.

Invocation permissions

An IAM resource policy gives permissions to invoke the function.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

28

An *IAM resource policy* tells the Lambda service which push event sources have permissions to invoke the Lambda function. Resource policies also make it easy to grant access to the Lambda function across AWS accounts. For example, suppose that you need an Amazon S3 bucket in account A to invoke your Lambda function in account B. You can create a resource policy that allows account A to invoke the function in account B.

Notice that the IAM resource policy is on the bucket, not on the Lambda function.

For more information, see “Using Resource-Based Policies for AWS Lambda” at <https://docs.aws.amazon.com/lambda/latest/dg/access-control-resource-based.html>.

Example resource (function) policy

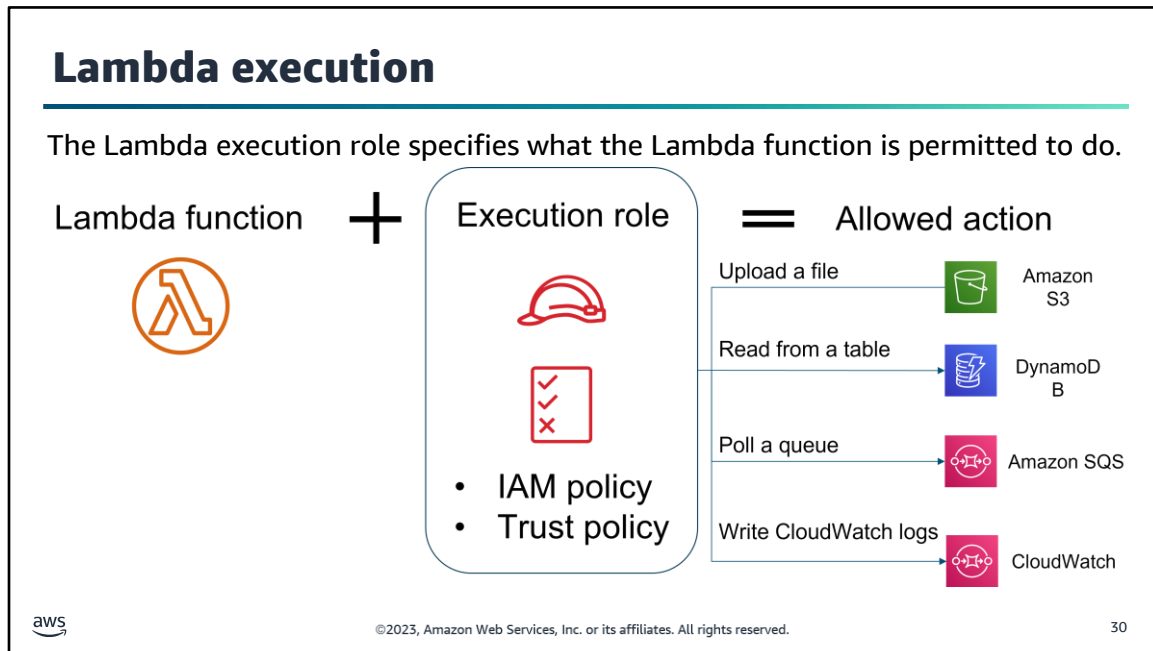
```
{
  "version": "2012-10-17",
  "id": "default",
  "statement": [
    {
      "sid": "lambda-fd269e28-988b-4d2b-96ae-eabcd7dc399c",
      "effect": "Allow",
      "principal": {
        "Service": "s3.amazonaws.com"
      },
      "action": "lambda:InvokeFunction",
      "resource": "arn:aws:lambda:function:myFirstFunction",
      "condition": {
        "ArnLike": {
          "AWS:SourceARN": "arn:aws:s3:::myBucket1"
        }
      }
    }
  ]
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

29

In this example, this resource policy on an S3 bucket gives Amazon S3 permissions to invoke the Lambda function that is called *myFirstFunction*.



The *Lambda execution role* grants your Lambda function permissions to access AWS services and resources. You select or create the execution role when you create a Lambda function.

Two policies are attached to the execution role:

- IAM policy* – Defines the actions that the Lambda function can take on another AWS service or resource, such as writing to a DynamoDB table
- Trust policy* – Allows the Lambda service to assume the execution role

To grant permissions to Lambda to use the AssumeRole action, you must have permissions for the *iam:PassRole* action.

Polling event sources also need the execution role to give the Lambda function permissions to read from the queue or stream.

The Lambda function also needs the execution role to poll event sources to be able to read from the queue or stream.

For more information, see “Lambda execution role” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html>.

Execution role example

IAM policy

```
{
  "Effect": "Allow",
  "Action": [
    "logs:CreateLogGroup",
    "logs:CreateLogStream",
    "logs:PutLogEvents"
  ],
  "Resource": "arn:aws:logs:*:*:*"
}
```

Trust policy

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "lambda.amazonaws.com"
  },
  "Action": "sts:AssumeRole"
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

IAM policies are associated with a role that another entity, such as Lambda or another resource, can assume. Policies aren't directly attached to a Lambda function.

Some examples of the relevant policy lines for a Lambda execution role include:

- In the example, the IAM policy allows your Lambda function to perform some CloudWatch Logs actions. These actions include creating a log group and log stream, and writing to the log stream.
- In the example, the trust policy gives the Lambda service permissions to assume the role and invoke the Lambda function on your behalf.

Adding permissions by using the AWS CLI

Give Amazon SNS permissions to invoke a function

```
aws lambda add-permission \  
--function-name my-function \  
--action lambda:InvokeFunction \  
--statement-id sns \  
--principal sns.amazonaws.com
```

Output

```
{  
  "Statement":  
  {  
    "Sid": "sns",  
    "Effect": "Allow",  
    "Principal": {  
      "Service": "sns.amazonaws.com"  
    },  
    "Action": "lambda:InvokeFunction",  
    "Resource": "arn:aws:lambda:us-east-2:123456789012:function:my-function"  
  }  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

32

This example shows the AWS Command Line Interface (AWS CLI) code to give Amazon SNS permissions to invoke a function and shows the IAM policy statement that results.

Section 5 key takeaways



- Lambda functions require permissions to be invoked and permissions to interact with other services.
- Both types of permissions are defined by using IAM or trust policies.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

33

Some key takeaways from this section of the module include:

- Lambda functions require permissions to be invoked and permissions to interact with other services.
- Both types of permissions are defined by using IAM or trust policies.



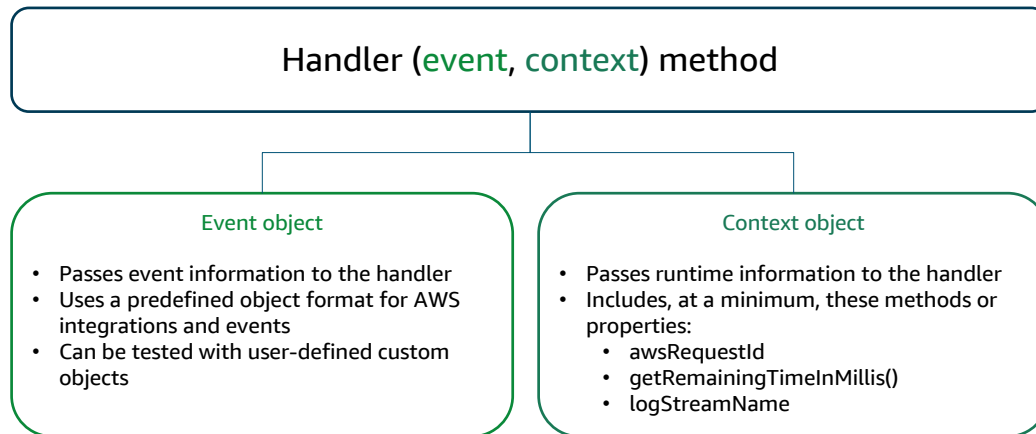
Section 6: Authoring and configuring Lambda functions

Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 34

Section 6: Authoring and configuring Lambda functions.

The function handler



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

35

When you create a Lambda function, you specify the function handler. The Lambda *function handler* is the entry point that Lambda calls to start running your Lambda function. The handler method always takes two objects: the event object and the context object.

- *Event object* – Provides information about the event that invoked the Lambda function. This information could be a predefined object that an AWS service generates, or it could be a custom user-defined object. A user-defined object could be a serializable string, such as a plain old Java object (POJO) or a JSON stream. The contents of the event object include all the data and metadata that your Lambda function needs to drive its logic. The contents and structure of the event object vary, depending on which event source created it. For example, an event that API Gateway created contains details that are related to the HTTPS request that the API client made. The details might include path, query string, and request body. However, an event that Amazon S3 created would include details about the bucket and the new object.
- *Context object* – AWS generates a context object and provides runtime information. The context object enables your function code to interact with the Lambda runtime environment. The contents and structure of the context object vary based on the language runtime that your Lambda function uses. However, at a minimum, the context object will contain:
 - *awsRequestId* – This property is used to track specific invocations of a Lambda function (important for error reporting or when contacting AWS Support).
 - *logStreamName* – This is the CloudWatch log stream that your log statements will be sent to.
 - *getRemainingTimeInMillis()* – This method returns the number of milliseconds that remain before running your function times out.

Lambda function handler example



This example is a Lambda function that is written in Python. It defines a handler function that is called `lambda_handler`. The function returns a message that contains data from the event that it received as input.

Notice that each language has its own requirements for how a function handler can be defined and referenced within the deployment package.

For more examples and details about how to create Lambda functions in your language of choice, see “Lambda Sample Applications” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-samples.html> in the AWS Lambda Developer Guide and individual sections for working with each runtime.

Example testing from the console

Create test event

Saved test event

test

```
1 {  
2   "first_name": "John",  
3   "last_name": "Smith",  
4 }
```

Review results

Tools Window Test Deploy Changes deployed

Execution results × lambda_function ×

Execution results Status: Succeeded Max memory used: 47 MB Time: 1.51 ms

Test Event Name
test

Response
{
 "message": "Hello John Smith!"
}

Function Logs
START RequestId: b5e4784b-94b4-4d3f-93c8-afdbe45fa213 Version: \$LATEST
END RequestId: b5e4784b-94b4-4d3f-93c8-afdbe45fa213
REPORT RequestId: b5e4784b-94b4-4d3f-93c8-afdbe45fa213 Duration: 1.51 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory

Request ID
b5e4784b-94b4-4d3f-93c8-afdbe45fa213



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

37

This image shows what the example from the previous slide looks like if you write and test it by using the Lambda console.

Performance-related configurations

Configuration	Description
Memory	Set the amount of memory and proportional CPU that is allocated to the function. Lambda allocates CPU power linearly in proportion to the memory that you set. Values from 128 MB–10,240 MB.
Timeout	Choose the maximum amount of time to let a function run before ending it if it has not completed. Values from 1 second up to 15 minutes.
Concurrency	Number of invocations of a function that can run at the same time. By default, you can have 1,000 concurrency invocations per Region for an account. This limit is a soft limit. Per function, you can set a limit to prevent overwhelming a downstream system or to reserve capacity from within the account pool for the function.
Provisioned concurrency	Number of Lambda environments to <i>keep warm</i> . When Lambda creates the temporary environment for your function, it will attempt to reuse it for additional invocations. This approach saves startup time. When Lambda must initialize new environments to run a function, an invocation might get a <i>cold start</i> , which creates startup latency. Use provisioned concurrency to avoid cold starts. Provisioned concurrency is priced separately.
Monitoring and operations	Settings to enable X-Ray (active tracing) and CloudWatch Lambda Insights (collect and aggregate Lambda function runtime performance metrics and logs).



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

38

Memory and timeout are configurations that determine how your Lambda function performs. These configurations affect your billing. With Lambda, you are charged based on the number of requests for your functions (the total number of requests across all your functions). You are also charged based on the duration (the time it takes for your code to run). The price depends on the amount of memory that you allocate to your function.

- **Memory** – You specify the amount of memory that you want to allocate to your Lambda function. Lambda then allocates CPU power that is proportional to the memory. Lambda is priced so that the cost per 1 millisecond of function duration increases as the memory configuration increases.
- **Timeout** – You can control the maximum duration of your function by using the timeout configuration. Using a timeout can prevent higher costs that come from long-running functions. You must find the right balance between not letting the function run too long and being able to finish under normal circumstances.

Follow these best practices:

- **Test the performance of your Lambda function** to make sure that you choose the optimum memory size configuration. You can view the memory usage for your function in CloudWatch Logs.
- **Load-test your Lambda function** to analyze how long your function runs and determine the best timeout value. This information is important when your Lambda function makes network calls to resources that might not be able to handle the scaling of Lambda functions.

For more information on AWS Lambda, see:

- “Lambda Quotas” at <https://docs.aws.amazon.com/lambda/latest/dg/limits.html>.
- “AWS Lambda Pricing” at <https://aws.amazon.com/lambda/pricing/>.
- “Configuring reserved concurrency” at <https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>.
- “Using AWS Lambda with AWS X-Ray” at <https://docs.aws.amazon.com/lambda/latest/dg/services-xray.html>.
- “Using Lambda Insights in Amazon CloudWatch” at <https://docs.aws.amazon.com/lambda/latest/dg/monitoring-insights.html>.

Resource-related configurations

Configuration	Description
Triggers	Event sources that invoke a function.
Permissions	The resources that have permissions to invoke the function and the permissions that the function has to interact with other resources.
Destinations	An SNS topic, SQS queue, other Lambda function, or EventBridge event bus. It receives invocation records from a function when it is successful (on success) or when it fails (on failure).
Asynchronous invocation	Settings for number of retry attempts on async invocations (0–2) and how long to keep an event waiting to be invoked (up to 6 hours). Also, configuration of a dead-letter queue for functions that continue to fail.
VPC	Settings to enable your Lambda function to access resources in a custom VPC. A custom VPC defines a private network of resources, such as databases, cache instances, or internal services.
State machines	Step Functions state machines that can invoke your function directly in at least one of the steps.
Database proxies	Settings that are used when connecting to an Amazon Relational Database Service (Amazon RDS) instance. These settings are used to set up a database proxy to manage connection pooling for database connections.
File systems	Settings to connect an Amazon Elastic File System (Amazon EFS) file system so that the function can access the file system at runtime.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

39

In addition to the resource-related configurations that you discussed so far (event sources, IAM permissions), Lambda offers more. Options are available for configuring how Lambda interacts with other AWS resources to adapt to different use cases. For example, you might need to connect to a virtual private cloud (VPC), an Amazon Relational Database Service (Amazon RDS) instance, or an Amazon Elastic File System (Amazon EFS) file system. You can also configure details that are related to event sources, such as destinations, and asynchronous invocation details. Both of these types are relevant to asynchronous event sources. Destinations can also be used for streaming event sources.

You can also configure Step Functions state machines that can directly invoke your function in one of their steps.

Code-related configurations

Configuration	Description
Runtime	Runtime that the function will use or language that the code will be written in. Choose from supported runtimes that are listed in the AWS Lambda Developer Guide or use a custom runtime.
Environment variables	Key-value pairs that are accessible from your function code. Environment variables are useful to store configuration settings without the need to change function code.
Tags	Labels that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Code signing	Option to ensure that code has been signed by an approved source and has not been altered.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

40

For more information about supported runtimes, see “Lambda Runtimes” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtimes.html>.

Design a Lambda function: Best practices for a Lambda environment

- Treat functions as stateless.
- Include only what you need.
- Reuse the temporary runtime environment.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

41

Some best practices to follow when you design your Lambda function are listed:

- *Treat functions as stateless.* No information about state should be saved within the context of the function itself. Because your functions exist only when work must be done, it's important for serverless applications to treat each function as stateless.
- *Include only what you need.*
 - Minimize both the size and the dependencies of your deployment package to reduce the startup time for your function. For example, choose only the modules that you need, such as the DynamoDB and Amazon S3 SDK modules, and Lambda core libraries. Don't include an entire AWS SDK library in your deployment package.
 - Reduce the time that it takes Lambda to unpack deployment packages that are authored in Java.
 - Minimize the complexity of your dependencies. Choose simpler Java dependency injection (IoC) frameworks. For example, choose Dagger or Guice instead of more complex frameworks like Spring.
- *Reuse the temporary runtime environment to improve the performance of your function.* The temporary runtime environment initializes any external dependencies of your Lambda function code. Make sure that any externalized configuration or dependencies that your code retrieves are stored and referenced locally after the initial run. Limit reinitializing variables and objects on every invocation. Keep alive and reuse connections—such as HTTP, database, or others—that were established during a previous invocation.

Design a Lambda function: Best practices for writing code

- Separate the core business logic (outside of the handler method).
- Write modular functions.
- Include logging statements.
- Include results information.
- Use environment variables.
- Avoid recursive code.
- Don't call one function from another.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

42

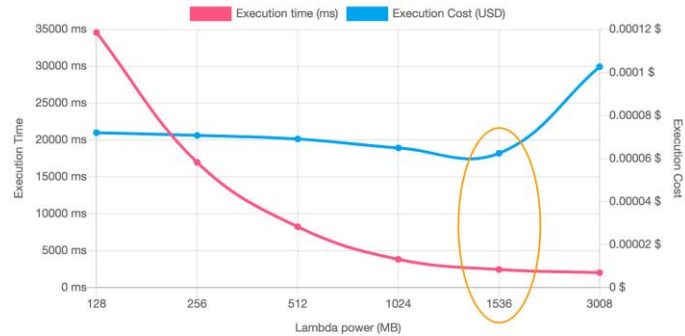
Some best practices to follow when you are writing code are listed:

- *Separate the core business logic from the handler method.* This practice makes your code more portable, and you can target unit tests at your code without concern about the configuration of the function.
- *Write modular functions.* Create single-purpose functions.
- *Include logging statements.* Lambda functions can and should include logging statements that are written to CloudWatch.
- *Include results information.* Functions must give information to Lambda about the results of their run.
- *Use environment variables.* With environment variables, you can pass operational parameters and configuration settings to your function without making changes to the code itself. For example, if you are writing to an S3 bucket, configure the bucket name as an environment variable instead of hardcoding the bucket name. You can also use environment variables to store sensitive information that the function requires.
- *Avoid using recursive code.* Avoid a situation where a function calls itself. In this circumstance, it might continue to spawn new invocations that would cause you to lose control of your concurrency.
- *Don't call one function from another.* Instead use destinations or orchestrate with Step Functions.

Tuning your function

- Higher memory configurations have a higher per-1-millisecond cost but might decrease duration costs and concurrency needs.
- Find the balance that optimizes for speed and cost.

Example results for two CPU-intensive functions, which become cheaper **and** faster with more power



Source: <https://github.com/alexcasalboni/aws-lambda-power-tuning>



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

43

Higher memory configurations have a higher per-1-millisecond cost, but might decrease duration costs and concurrency needs. For example, with applications that are CPU intensive, increasing the memory (which provides more CPU) can shorten the duration of the function. Therefore, it might be cheaper to run it at the higher memory level.

Use the power tuning utility to find the balance that suits your application needs. For more information about power tuning, see “aws-lambda-power-tuning” at <https://github.com/alexcasalboni/aws-lambda-power-tuning>.

Section 6 key takeaways



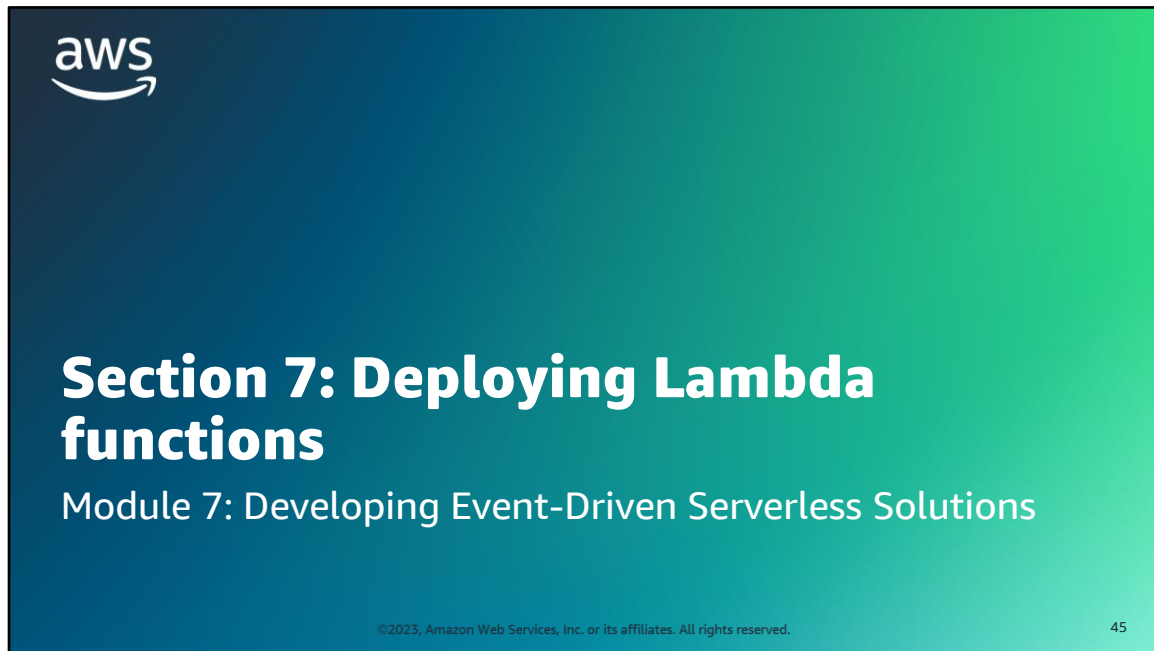
- Lambda supports different languages and runtimes.
- The function handler is the entry point into the Lambda function.
- You can configure attributes including the amount of memory and function timeout.
- Follow best coding practices to create efficient Lambda functions.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

44

Some key takeaways from this section of the module include the following:

- Lambda supports several different languages and runtimes.
- The function handler is the entry point into the Lambda function.
- You can configure the amount of memory and function timeout.
- Follow best coding practices to create efficient Lambda functions.



Section 7: Deploying Lambda functions: Overview.

Deployment options

.zip archive

- Choose a **runtime** when you **create the function**.
- Compress files for application code and dependencies, and **upload them to Lambda**.

Container image

- Choose a **runtime and linux distribution** when **creating the image**.
- Package your code and dependencies as a **container image**.
- **Upload** the image to your **container registry** that is hosted on Amazon Elastic Container Registry (Amazon ECR).



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

46

To deploy a Lambda function with the Lambda API, command-line tools, or the AWS SDKs, you must create a Lambda function deployment package. You also must create a deployment package if your function uses a compiled language or to add dependencies to your function.

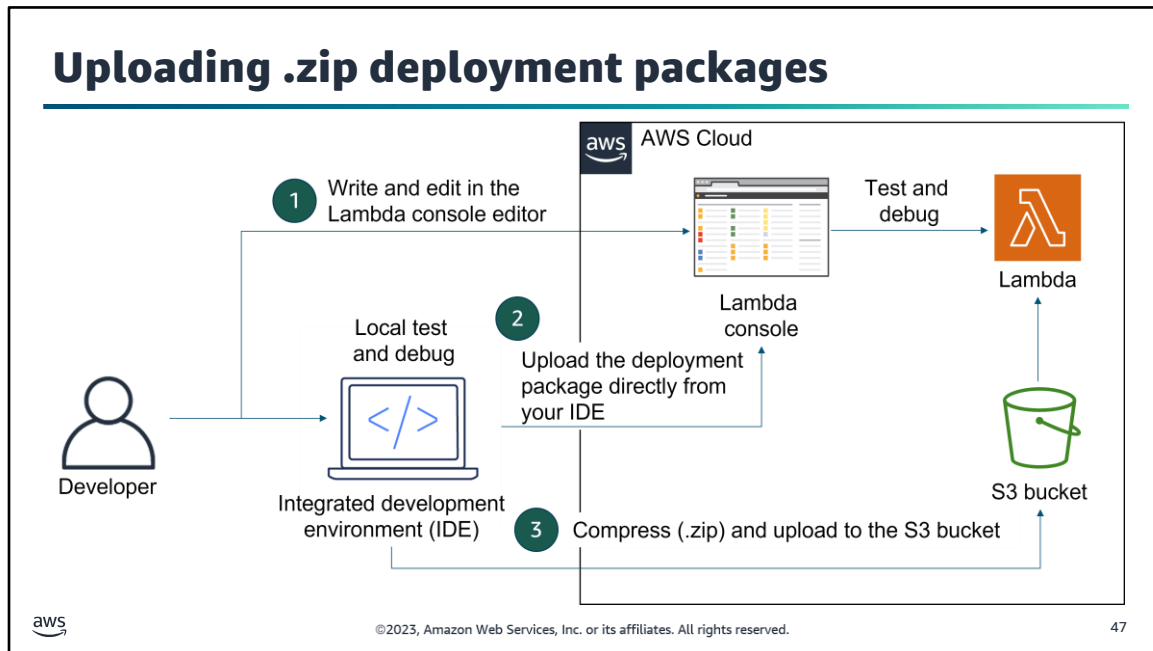
You can create two types of deployment packages, .zip archives or container images.

If you use a .zip archive, you choose a runtime when you use the Lambda console or a toolkit to create a function. When you author your function, Lambda will automatically create the .zip file of your code.

You can also use a container image for your deployment package. Thus, it is possible to use familiar container tooling, workflows, and dependencies to build applications that are based on Lambda.

For more information, see the following resources:

- “Lambda Runtimes” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtimes.html>.
- “Lambda Deployment Packages” at <https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-package.html>.
- “Working with Lambda container images” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-images.html>.

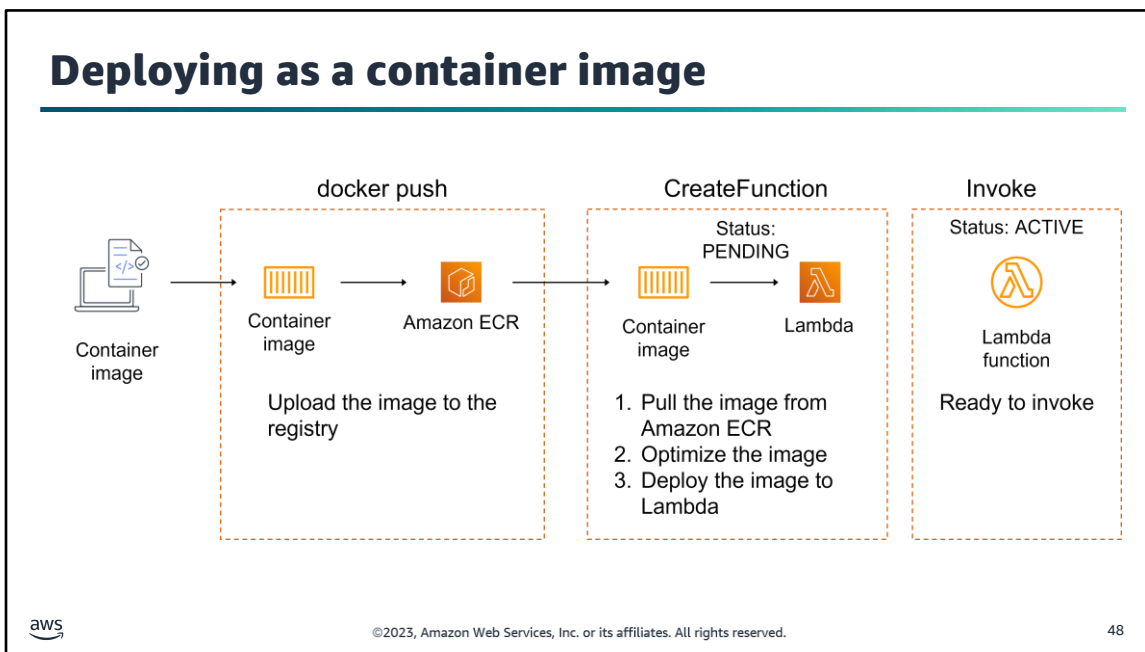


You can use the console or use your integrated development environment (IDE) to author Lambda functions. For .zip file deployment packages, you have three options for providing the deployment package to the Lambda service:

- **Lambda console editor** – If your code does not require custom libraries (other than the AWS SDK), then you can edit your code inline through the AWS Management Console. The console compresses your code (with the relevant configuration information) into a deployment package that the Lambda service can run. This deployment choice is ideal for simple scenarios.
- **Upload the deployment package directly from your IDE** – In most scenarios where your code requires custom libraries, you can create your deployment package in your IDE. Then, you can upload it through the Lambda console.
- **Compress and upload to an S3 bucket** – You can also upload your deployment package to an S3 bucket and provide the S3 bucket, object key, and optional version for the object. Lambda will load your code directly from Amazon S3.

The deployment packages have the following limits:

- 50 MB (compressed, for direct upload)
- 250 MB (not compressed, including layers)
- 3 MB (using the console editor)
- 512 KB maximum for an individual file



When you deploy a container image, first deploy the image to Amazon ECR by using the `docker push` command. During the Lambda create or update process, the Lambda service pulls the image from Amazon ECR, optimizes the image for use, and deploys the image to the Lambda service. After this process and any other configuration processes are complete, the Lambda function is then in Active status and ready to be invoked.

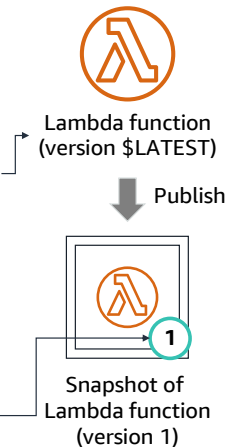
Versioning

- Immutable copies of Lambda function code and configuration
- Create: \$LATEST version

`arn:aws:lambda:aws-region:acct-id:function:helloWorld:$LATEST`

- Publish: Snapshot copy of \$LATEST
- Each version has its own Amazon Resource Name (ARN) with a new sequential version number

`arn:aws:lambda:aws-region:acct-id:function:helloWorld:1`



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

49

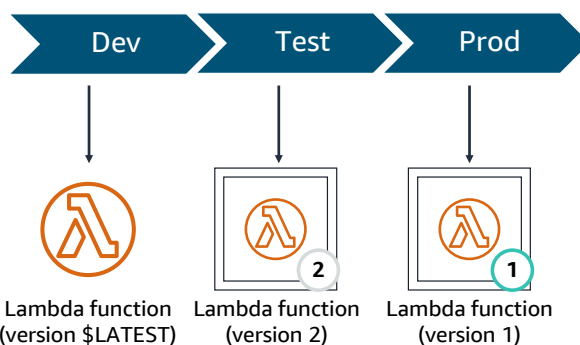
Versions are immutable copies of the code and configuration of your Lambda function. You use versioning to publish one or more versions of your Lambda function. As a result, you can work with different variations of your Lambda function in your development workflow, such as development, beta, and production.

When you create a Lambda function, only one version exists—the \$LATEST version. You can refer to this function by using its Amazon Resource Name (ARN). When you publish a new version, Lambda makes a snapshot copy of the \$LATEST version to create the new version. The new version has a unique ARN that includes a new sequential version number.

For more information about versioning, see “Lambda Function Versions” at <https://docs.aws.amazon.com/lambda/latest/dg/versioning-intro.html>.

Aliases

- Mutable pointers to versions
- Each alias has its own ARN



Lambda function test alias:

```
arn:aws:lambda:aws-region:acct-id:function:helloWorld:test
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

50

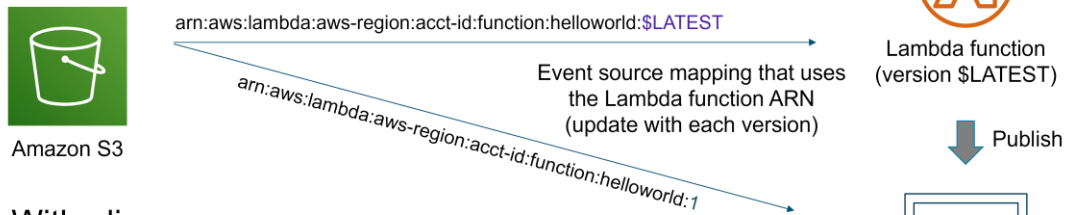
You can create aliases for your Lambda function. Conceptually, an *alias* is a pointer to a specific Lambda function version. You can use the alias in the ARN to reference the Lambda function version that is currently associated with that alias. Using an alias makes it easy to promote or roll back versions without changing any code. Aliases abstract the need to know which version is being referenced.

In this example, the test alias points to version 2 of the Lambda function.

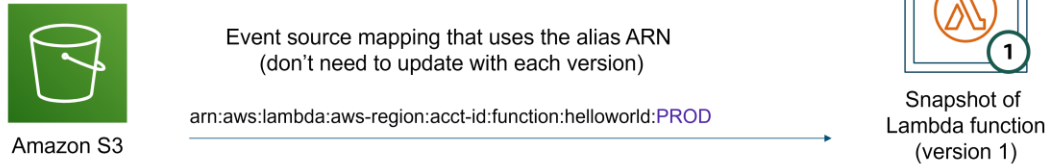
For more information about aliases, see “Lambda Function Aliases” at <https://docs.aws.amazon.com/lambda/latest/dg/aliases-intro.html>.

Example of using versioning and aliases

Without aliases:



With aliases:



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

51

For example, suppose that Amazon S3 is the event source that invokes your Lambda function when new objects are created in a bucket. When Amazon S3 is your event source, you store the information for event source mapping in the configuration for bucket notifications. In that configuration, you can identify the Lambda function ARN that Amazon S3 can invoke. However, in this case, you must update the notification configuration. It is necessary so that Amazon S3 will invoke the correct version each time you publish a new version of your Lambda function.

Instead of specifying the function ARN, you can specify an alias ARN in the notification configuration (for example, you can specify the PROD alias ARN). As you promote new versions of your Lambda function into production, you only must update the PROD alias to point to the latest stable version. You don't need to update the notification configuration in Amazon S3.

Custom runtimes

- With custom runtimes you can use **other languages** in Lambda.
- Lambda provides a **runtime interface** for getting events and sending responses.
- The **bootstrap file** is the runtime's entry point.
- You can deploy a custom runtime next to your function code or **in a layer**.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

52

With custom runtimes, you can use other languages in Lambda. The Lambda invocation environment provides a runtime interface for getting events and sending responses.

The bootstrap file is the runtime's entry point, and you can deploy a custom runtime next to your function code or in a layer.

For more information about custom runtimes, see “Custom AWS Lambda Runtimes” at <https://docs.aws.amazon.com/lambda/latest/dg/runtimes-custom.html>.

Lambda layers

Centrally manage code and data that are shared across multiple functions

- Reduce the size of deployment packages
- Speed up deployments
- Limits:
 - Up to five layers
 - 250 MB



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

53

Even if you are using a standard runtime, when you build serverless applications, it is common to have code that is shared across Lambda functions. It can be custom code that more than one function uses, or a standard library that you add to simplify the implementation of your business logic.

Previously, you needed to package and deploy this shared code together with all the functions that used it. Now, you can configure your Lambda function to include additional code and content as layers. A *layer* is a .zip archive that contains libraries, a custom runtime, or other dependencies. With layers, you can use libraries in your function without needing to include them in your deployment package.

It is a best practice to have smaller deployment packages and to share common dependencies with layers. You can use layers to keep your deployment package small, which makes development easier. You can avoid errors that can occur when you install and package dependencies with your function code. For Node.js, Python, and Ruby functions, if you keep your deployment package under 3 MB, you can develop your function code in the Lambda console.

A function can use up to five layers at a time. The total extracted size of the function and all layers cannot exceed the extracted deployment package size limit of 250 MB.

AWS published a public layer that includes NumPy and SciPy, which are scientific libraries for Python. This layer can help you with data processing and machine learning applications. For information about how to use this layer, see “New for AWS Lambda – Use Any Programming Language and Share Common Components” at <https://aws.amazon.com/blogs/aws/new-for-aws-lambda-use-any-programming-language-and-share-common-components/> on the AWS News Blog.

For more information about Lambda layers, see “Lambda layers” at <https://docs.aws.amazon.com/lambda/latest/dg/configuration-layers.html>.

Example deploying from the AWS CLI

Create function

```
aws lambda create-function \
--function-name my-function \
--runtime nodejs10.x \
--zip-file fileb://my-function.zip \
--handler my-function.handler \
--role arn:aws:iam::123456789012:role/service-role/MyTestFunction-role-tges6bf4
```

Output

```
{
  "TracingConfig": {
    "Mode": "PassThrough"
  },
  "CodeSha256": "PFn4S+er27qk+UuZSTKEQfNKG/XNn7QJs90mJgq6oH8=",
  "FunctionName": "my-function",
  "CodeSize": 308,
  "RevisionId": "873282ed-4cd3-4dc8-a069-d0c647e470c6",
  "MemorySize": 128,
  "FunctionArn": "arn:aws:lambda:us-west-2:123456789012:function:my-function",
  "Version": "$LATEST",
  "Role": "arn:aws:iam::123456789012:role/service-role/MyTestFunction-role-zgur6bf4",
  "Timeout": 3,
  "LastModified": "2019-08-14T22:26:11.234+0000",
  "Handler": "my-function.handler",
  "Runtime": "nodejs10.x",
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

54

The code snippets on this slide illustrate how you would use the AWS CLI to deploy a function.

Example of updating a function from the AWS CLI

Update function

```
aws lambda update-function-code \  
--function-name my-function \  
--zip-file fileb://my-function.zip
```

Output

```
{  
  "FunctionName": "my-function",  
  "LastModified": "2019-09-26T20:28:40.438+0000",  
  "RevisionId": "e52502d4-9320-4688-9cd6-152a6ab7490d",  
  "MemorySize": 256,  
  "Version": "$LATEST",  
  "Role": "arn:aws:iam::123456789012:role/service-role/my-function-role-uy319qq",  
  "Timeout": 3,  
  "Runtime": "nodejs10.x",  
  "TracingConfig": {  
    "Mode": "PassThrough"  
  },  
  "CodeSha256": "5tT2qgzYUHaqWR716pz2dpkn/0j1FrzJm1kidwoaCgk=",  
  "CodeSize": 304,  
  "FunctionArn": "arn:aws:lambda:us-west-2:123456789012:function:my-function",  
  "Handler": "index.handler"  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

55

The code snippets on this slide illustrate how you would update an existing function using the AWS CLI.

Section 7 key takeaways



- Lambda has built-in versioning so that you can create immutable versions of your code.
- You can use aliases to point at different versions of your Lambda function.
- Lambda provides popular runtimes, but you can create custom runtimes.
- Lambda layers are libraries of code that you create and which other Lambda functions can call.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

56

Some key takeaways from this section of the module include:

- Lambda has built-in versioning to allow creation of immutable versions of your code.
- You can use aliases to point at different versions of your Lambda function.
- Lambda provides popular runtimes, but you can create custom runtimes.
- Lambda Layers are libraries of code that you create and which other Lambda functions can call.



Section 8: Monitoring and debugging tools for application developers.

Modern applications challenge traditional monitoring approaches



Short-lived
resources



More devices,
services, and
data



Faster release
velocity



User
experience



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

58

With modern applications, and particularly with serverless ones, you must deal with short-lived resources. You also encounter many different services that are connected in a distributed way, and faster release velocity of independent components. The user experience has also become a more important consideration for the performance of your applications; for example, users don't tolerate slow connections. This type of monitoring requires a different approach than alerting on errors in one of the layers in your stack.

Amazon CloudWatch



Amazon
CloudWatch

CloudWatch collects monitoring and operational data in the form of logs, metrics, and events.

- Get a **unified view** of AWS resources, applications, and services
- Review logs as a **flow of events ordered by time**
- **Query** log data interactively with **CloudWatch Logs Insights**
- Use **default operational metrics** from AWS services
- **Alarm** based on metric thresholds and initiate **automated actions**



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

59

Amazon CloudWatch collects monitoring and operational data in the form of logs, metrics, and events.

With CloudWatch you can do the following:

- Get a unified view of AWS resources, applications, and services
- Review logs as a flow of events ordered by time
- Query log data interactively with CloudWatch Logs Insights
- Use default operational metrics from AWS services
- Alarm based on metric thresholds and initiate automated actions

Example of Lambda logging

Log group		Log stream	
CloudWatch > Log Groups > /aws/lambda/FAQ > 2020/08/27/[\$LATEST]05419d29f9014fc3b1634b1e9968ef28			
Time	Message	Log events	
19:35:32	START RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 Version: \$LATEST		
19:35:32	2020-08-27T19:35:32.553Z 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 INFO ...		
19:35:32	2020-08-27T19:35:32.575Z 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 INFO ...		
19:35:32	END RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937		
19:35:32	REPORT RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 Duration: 64.40 ms Billed Duration: 100 ms Memory Size: 128 MB Max Memory Used: 68 MB Init Duration: 162.85 ms		
19:35:36	START RequestId: a23cbea6-3053-4b51-8721-ff11aba24e33 Version: \$LATEST		
...			



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

60

Lambda logs all requests and results in a CloudWatch log group that is called `/aws/lambda/<function name>`. A one-to-one relationship exists between function and log group. The first time that the function runs, the group is created. If you set up provisioned concurrency, the log group is created when you create the provisioned concurrency. The log group is the entity that you would typically use for searching events within your Lambda logs.

The log group is made up of log streams that are identified by the date, **function version**, and a unique identifier. Lambda automatically creates new log streams within a group. They are based on internal factors that are related to how invocations are allocated to environments and the timing of invocations.

When you test functions from the Lambda console, log information is presented on the console to assist with debugging. The console highlights the success or failure of the invocation, and provides summary information from the log entries.

Each log stream contains a sequence of events that describe Lambda request details. Each request includes a unique **RequestId**, and a START event, END event, and REPORT event, which quickly tell you summary information about the Lambda request. The RequestId is the primary identifier that you will use to find events related to a particular invocation. The REPORT event is a quick way to see how long the function ran. It also shows the billing duration, the memory that is configured for the function, and the memory that it used. The “Init” duration in this example tells you that this invocation was a cold start.

For more information about CloudWatch Logs, see “What Is Amazon CloudWatch Logs?” at <https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html>.

For more information about accessing CloudWatch Logs for Lambda, see “Accessing Amazon CloudWatch logs for AWS Lambda” at <https://docs.aws.amazon.com/lambda/latest/dg/monitoring-cloudwatchlogs.html>.

AWS X-Ray



AWS X-Ray

X-Ray traces requests that travel through your application and shows a map of your application's underlying component.

- **One-click** enablement for **Lambda functions** and integration with other AWS services
- **X-Ray SDK** to capture metadata for **API calls** that are made to AWS services with the **AWS SDK**
- Visual **detection** of **latency distribution** and quick **isolation** of **outliers** and trends
- Easier **debugging** of application code
- **Filtering** and **grouping** of requests by error type



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

61

X-Ray traces requests that travel through your application, and shows a map of your application's underlying components.

X-Ray has the following features:

- One-click enablement for Lambda functions and integration with other AWS services
- X-Ray SDK to capture metadata for API calls that are made to AWS services with the AWS SDK
- Visual detection of latency distribution and quick isolation of outliers and trends
- Easier debugging of application code
- Filtering and grouping of requests by error type

Parts of an X-Ray trace

Element	Description
Segments	Data about the work done by compute resources that run your application, including: • Resource name • Details about the request and the work that is done • Subsegments
Subsegments	Breakdown of segment data into greater detail, including: • More granular timing • Details about downstream calls • Inferred segments for downstream services • Annotations and metadata
Annotations	Key-value pairs that can be indexed and used with filter expressions to group traces in the console for easier analysis
Metadata	Key-value pairs of any type that are not indexed but can be used to store data in the trace that you won't use for searching traces



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

62

A *segment* provides the resource's name, details about the request, and details about the work that is done.

A segment can break down the data about the work that is done into *subsegments*. Subsegments provide more granular timing information and details about downstream calls that your application made to fulfill the original request. A subsegment can contain additional details about a call to an AWS service, external HTTP API, or SQL database. You can even define arbitrary subsegments to instrument specific functions or lines of code in your application.

Annotations include key-value pairs that can be indexed and used with filter expressions to group traces in the console for easier analysis.

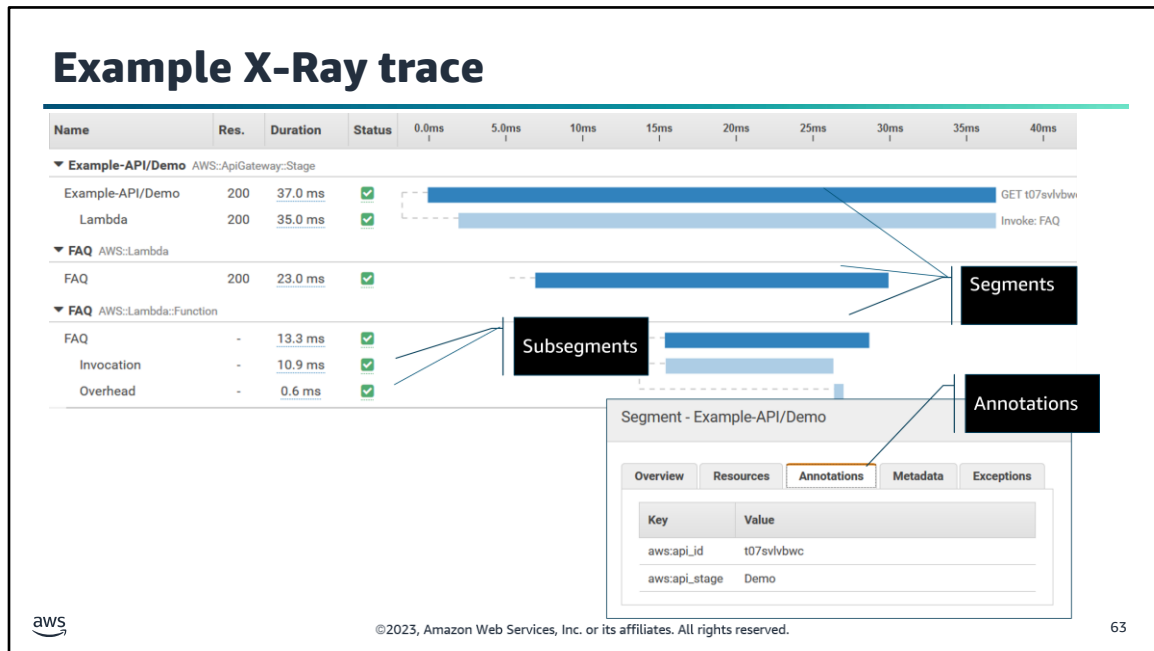
Metadata includes key-value pairs of any type that are not indexed but can be used to store data in the trace that you won't use for searching traces

For information about segments, see “AWS X-Ray concepts” at <https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html>.

For information about X-Ray segment documents, see “AWS X-Ray segment documents” at <https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html>.

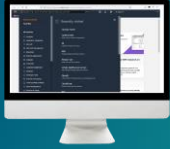
For information about annotations and metadata, see “AWS X-Ray concepts” at <https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html>.

This additional data makes it easier to use filter expressions and look at your traces in logical groups. For more information about using filter expressions to search for traces in the console, see “Using filter expressions to search for traces in the console” at <https://docs.aws.amazon.com/xray/latest/devguide/xray-console-filters.html>.



This diagram illustrates an X-Ray trace that is generated when an API Gateway API invokes a Lambda function.

Demonstration: Using X-Ray with Lambda



aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

64

There is a video demonstration available for this topic. You can find this video within the module 7 section of the course with the title: **Demo Using X-Ray with Lambda.**

If you are unable to locate this video demonstration please reach out to your educator for assistance.

Section 8 key takeaways



- Modern applications require more than traditional operations monitoring.
- CloudWatch provides built-in logging and metrics to monitor your Lambda functions.
- With X-Ray, you can trace requests across your application to find trouble spots quickly.

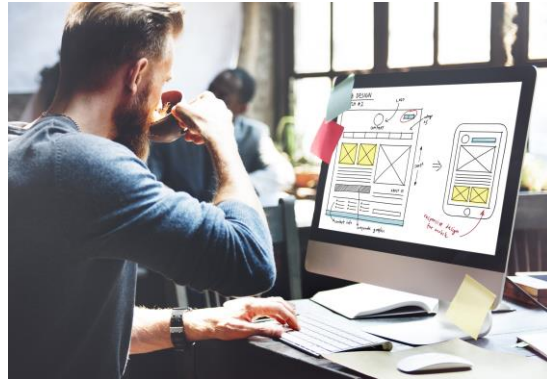
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

65

Some key takeaways from this section of the module include the following:

- Modern applications require more than traditional operations monitoring.
- CloudWatch provides built-in logging and metrics for monitoring your Lambda functions.
- With X-Ray, you can trace requests across your application to find trouble spots quickly.

Lab 7.1: Creating Lambda Functions Using the AWS SDK for Python



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

66

You will now complete Lab 7.1: Creating Lambda Functions Using the AWS SDK for Python.

Lab: Scenario

In this lab, you again play the role of Sofía. You replace the mock endpoints that you created in the previous lab with real endpoints so that the web application can connect to the database. You will use Lambda to make this connection between the REST API and the data that is stored in DynamoDB.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

67

In this lab, you again play the role of Sofía. You replace the mock endpoints that you created in the previous lab with real endpoints so that the web application can connect to the database. You will use Lambda to make this connection between the REST API and the data that is stored in DynamoDB.

Lab: Tasks

1. Preparing the development environment
2. Creating the `get_all_products` Lambda function
3. Configuring the REST API GET method to invoke the Lambda function
4. Creating the `create_report` Lambda function
5. Configuring the REST API POST method to invoke the Lambda function
6. Testing the integration by using the café website

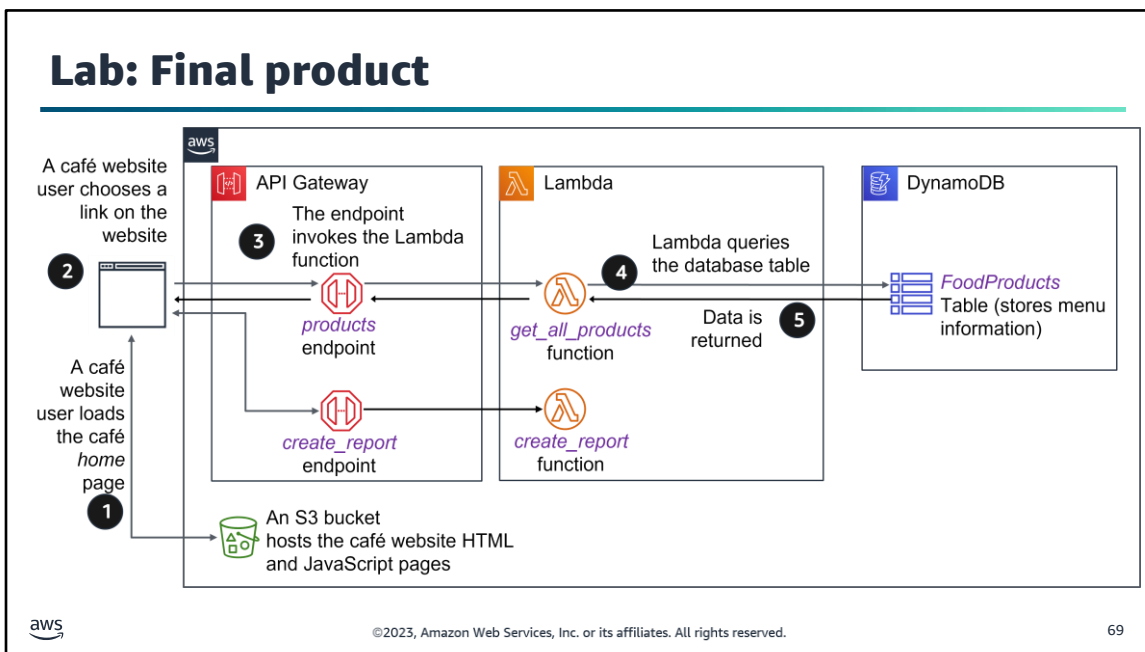


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

68

In this lab, you will complete the following tasks:

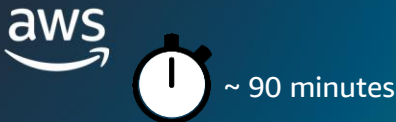
1. Preparing the development environment
2. Creating the `get_all_products` Lambda function
3. Configuring the REST API GET method to invoke the Lambda function
4. Creating the `create_report` Lambda function
5. Configuring the REST API POST method to invoke the Lambda function
6. Testing the integration by using the café website



The diagram summarizes what you will have built after you complete the lab.

When you complete the lab, the user loads the café home page and chooses a link to return a list of products on the website. Then, API Gateway invokes the `get_all_products` Lambda function. The function queries the DynamoDB table and returns the product's information.

If the user selects the option to create the report, the request reaches a different API Gateway endpoint that is integrated with the `create_report` function.



**Begin Lab 7.1:
Creating Lambda
Functions Using
the AWS SDK for
Python**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 70

It is now time to start the lab.

Lab debrief: Key takeaways

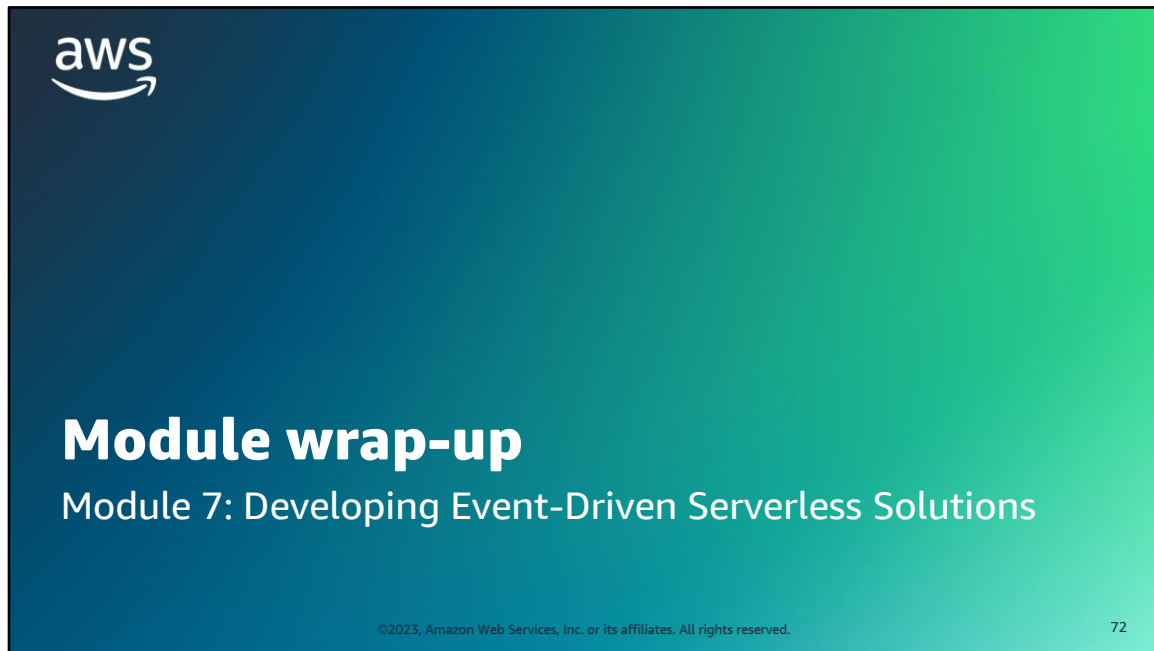


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

71

Your educator might choose to lead a conversation about the key takeaways from this lab after you have completed it.



It's now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to:

- Explain serverless computing
- Describe how Lambda works
- Recognize Lambda invocation models
- Identify how to use IAM to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why X-Ray is a critical developer tool
- Create Lambda functions



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

73

This module addressed the following topics:

- Explain serverless computing
- Describe how Lambda works
- Recognize Lambda invocation models
- Identify how to use IAM to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why X-Ray is a critical developer tool
- Create Lambda functions

To finish this module, complete the corresponding knowledge check.

It is simple to get started with Lambda, and the possibilities are broad for its use. However, it takes some practice to get used to *thinking serverless*. Keep the following key things in mind:

- **15-minute maximum duration:** If your function cannot routinely run in less than 15 minutes, you won't be able to put it into a function. Follow best practices for limiting the time that is required to run your function.
 - For more information on best practices, see “Best practices for working with AWS Lambda functions” at <https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html>.
- **Concurrency:** Lambda scales to meet demand up to the account or function limits. Use the function concurrency setting to manage the maximum concurrency to

avoid overwhelming downstream systems.

- **Cold start latency:** Test your functions in production-like conditions to determine whether the impact of cold starts affect your function's ability to respond as quickly as needed. Tune your function to shorten the startup time, and consider using provisioned concurrency to prevent cold starts.
 - For more information on cold start latency, see:
 - “AWS Lambda Execution Environment” at <https://docs.aws.amazon.com/lambda/latest/dg/runtimes-context.html>.
 - “Managing Concurrency for a Lambda Function” at <https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>.
- **Lambda and VPCs:** When you are working with VPC resources, you have additional considerations for your function. If you must access VPC resources, use the available resources to plan your configuration.
 - For more information on AWS Lambda and VPCs, see:
 - “Configuring a Lambda Function to Access Resources in a VPC” at <https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html>.
 - “Configuring Interface VPC Endpoints for Lambda” at <https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-endpoints.html>.

Complete the knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

74

It is now time to complete the knowledge check for this module.

Sample exam question

A developer is testing an application locally and has deployed it to Lambda. To remain under the package size limit, they did not include the dependencies in the deployment file. When testing the application remotely, the function does not run because of missing dependencies. Which approach would best resolve the issue?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Under the package size limit
- Dependencies



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

75

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Response choices

A developer is testing an application locally and has deployed it to Lambda. To remain **under the package size limit**, they did not include the **dependencies** in the deployment file. When testing the application remotely, the function does not run because of missing dependencies. Which approach would best resolve the issue?

Choice	Response
A	Use the Lambda console editor to update the code and include the missing dependencies.
B	Create an additional .zip file with the missing dependencies, and include the file in the original Lambda deployment package.
C	Add references to the missing dependencies in the Lambda function's environment variables.
D	Create a layer that contains the missing dependencies, and attach it to the Lambda function.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 76

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is D.

Choice	Response
A	Use the Lambda console editor to update the code and include the missing dependencies.
B	Create an additional .zip file with the missing dependencies, and include the file in the original Lambda deployment package.
C	Add references to the missing dependencies in the Lambda function's environment variables.
D	Create a layer that contains the missing dependencies, and attach it to the Lambda function.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

77

The correct answer is **D: Create a layer that contains the missing dependencies, and attach it to the Lambda function.**

Additional resources

- AWS Lambda Developer Guide:
 - [Best Practices for Working with AWS Lambda Functions](#)
 - [Using AWS Lambda with Other Services](#)
 - [AWS Lambda Execution Environment](#)
 - [Troubleshooting Issues in Lambda](#)
 - [Monitoring Functions in the AWS Lambda Console](#)
 - [Accessing Amazon CloudWatch logs for AWS Lambda](#)
 - [Using AWS Lambda with AWS X-Ray](#)
- Amazon CloudWatch Logs User Guide:
 - [Analyzing Log Data with CloudWatch Logs Insights](#)

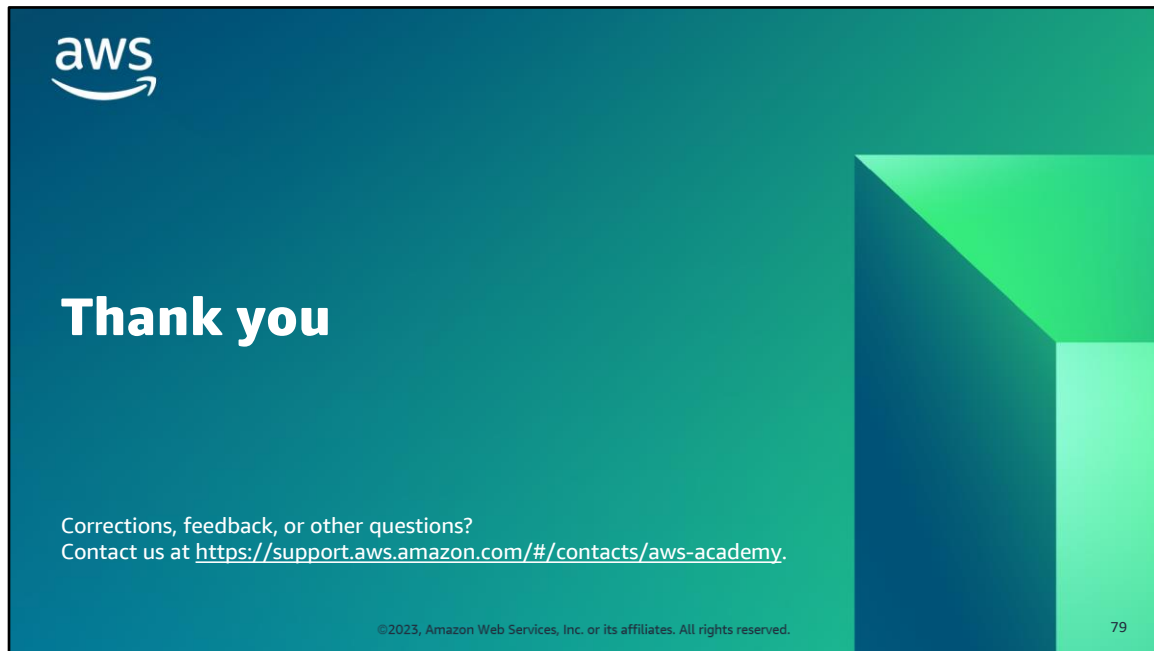


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

78

If you want to learn more about the topics covered in this module, you might find the following additional resources helpful:

- For more information in the AWS Lambda Developer Guide, see:
 - “Best Practices for Working with AWS Lambda Functions” at <https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html>.
 - “Using AWS Lambda with Other Services” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-services.html>.
 - “AWS Lambda Execution Environment” at <https://docs.aws.amazon.com/lambda/latest/dg/runtimes-context.html>.
 - “Troubleshooting Issues in Lambda” at <https://docs.aws.amazon.com/lambda/latest/dg/lambda-troubleshooting.html>.
 - “Monitoring Functions in the AWS Lambda Console” at <https://docs.aws.amazon.com/lambda/latest/dg/monitoring-functions-access-metrics.html>.
 - “Accessing Amazon CloudWatch Logs for AWS Lambda” at <https://docs.aws.amazon.com/lambda/latest/dg/monitoring-cloudwatchlogs.html>.
 - “Using AWS Lambda with AWS X-Ray” at <https://docs.aws.amazon.com/lambda/latest/dg/services-xray.html>.
- For more information in the Amazon CloudWatch Logs User Guide, see:
 - “Analyzing Log Data with CloudWatch Logs Insights” at <https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html>.



Thank you for completing this module.